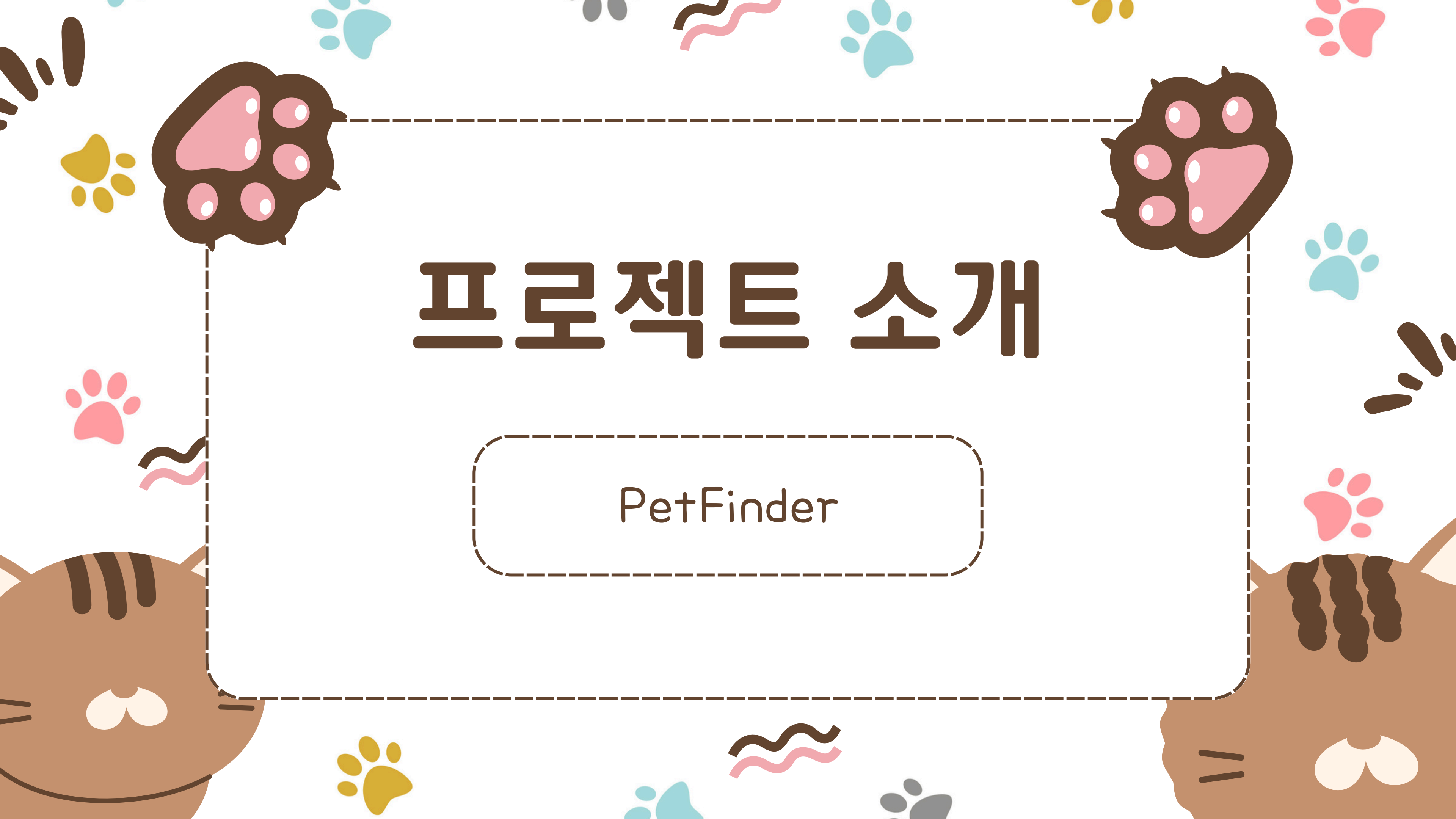




# Euronimals

ML세션

진웨이안(팀장), 김서연, 김예나, 최혜윤

The background is white with various colorful paw prints (pink, blue, yellow, grey) and stylized cat faces (brown with white whiskers and bell-shaped eyes) scattered around. A dashed brown line forms a rectangular frame around the central text.

# 프로젝트 소개

PetFinder

# 프로젝트 소개



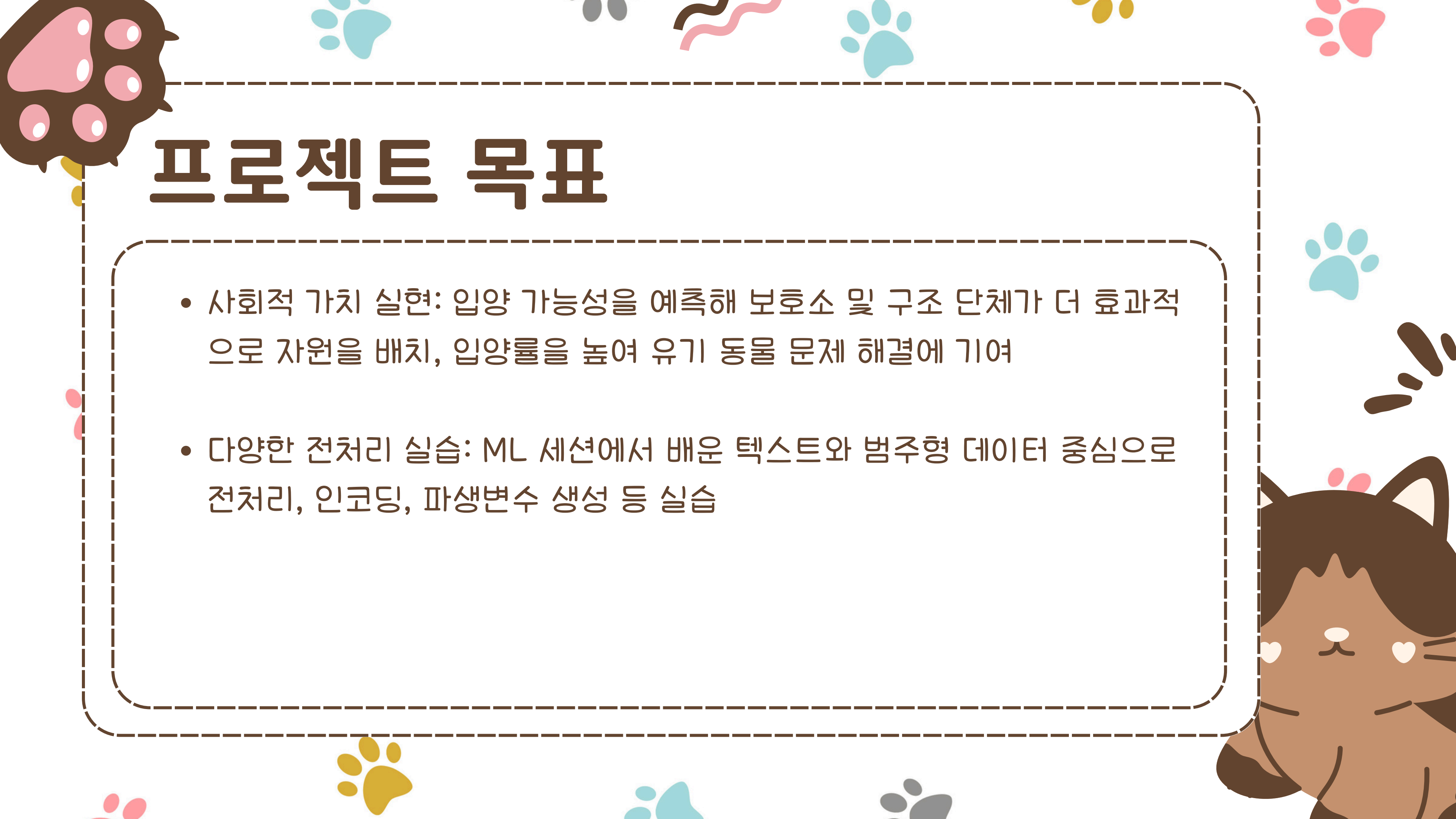
PETFINDER.MY · FEATURED CODE COMPETITION · 6 YEARS AGO

## PetFinder.my Adoption Prediction

How cute is that doggy in the shelter?

- PetFinder.my 플랫폼에 등록된 유기동물의 **입양 속도**를 예측





# 프로젝트 목표

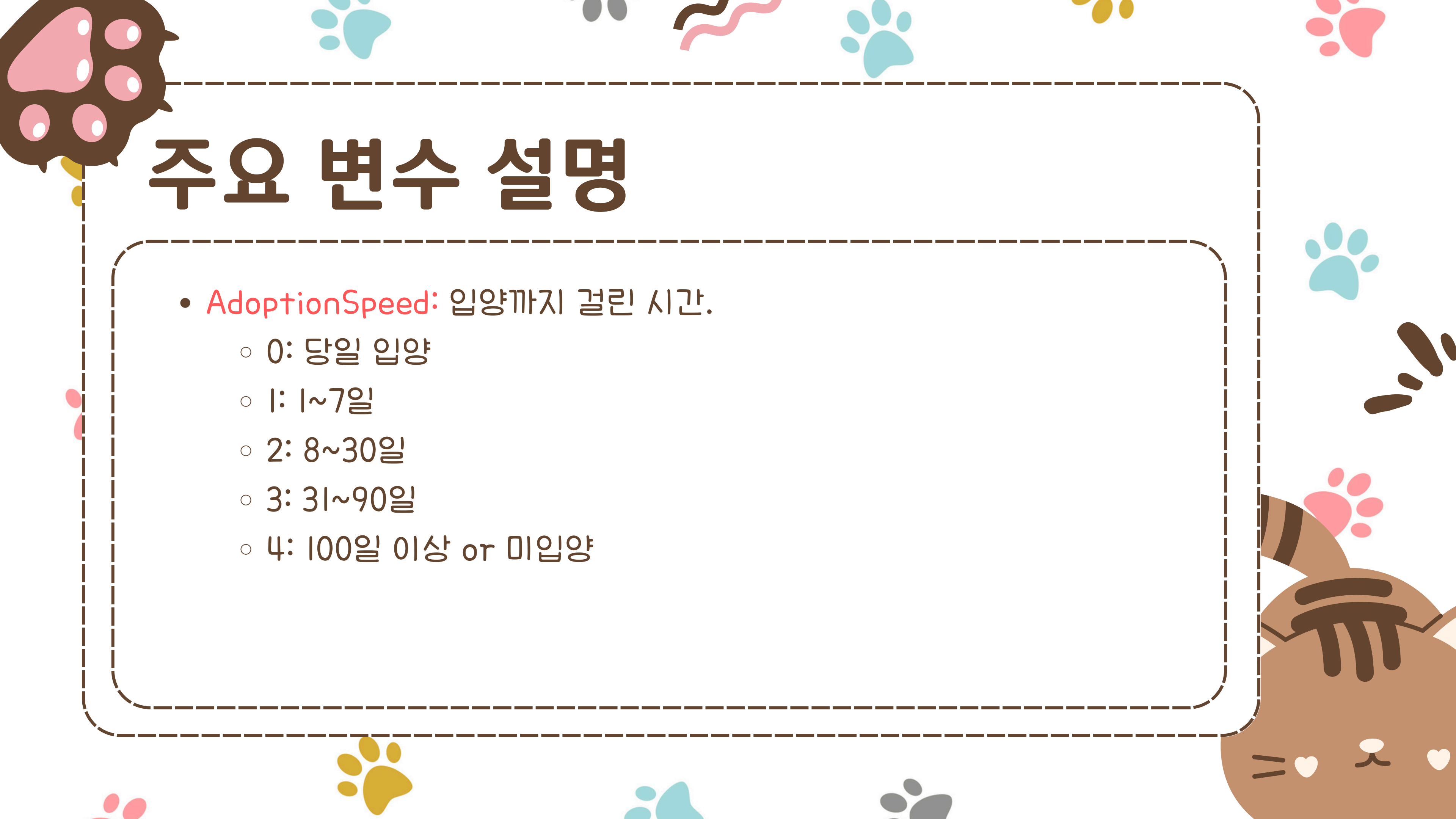
- 사회적 가치 실현: 입양 가능성을 예측해 보호소 및 구조 단체가 더 효과적으로 자원을 배치, 입양률을 높여 유기 동물 문제 해결에 기여
- 다양한 전처리 실습: ML 세션에서 배운 텍스트와 범주형 데이터 중심으로 전처리, 인코딩, 파생변수 생성 등 실습



# 데이터셋 구성

train.csv

- 약 15,000개의 유기동물 데이터
- 주요 컬럼 예: PetID, AdoptionSpeed, Type, Age, Breed1/2, Gender, Color1/2/3, MaturitySize, FurLength, Vaccinated, Dewormed, Sterilized, Health, Quantity, Fee, State, RescuerID, VideoAmt, PhotoAmt, Description 등



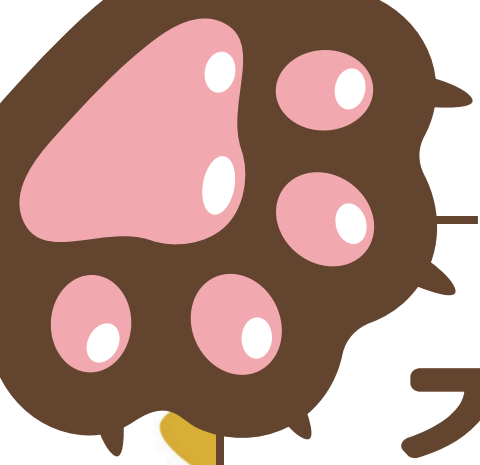
# 주요 변수 설명

- **AdoptionSpeed**: 입양까지 걸린 시간.
  - 0: 당일 입양
  - 1: 1~7일
  - 2: 8~30일
  - 3: 31~90일
  - 4: 100일 이상 or 미입양

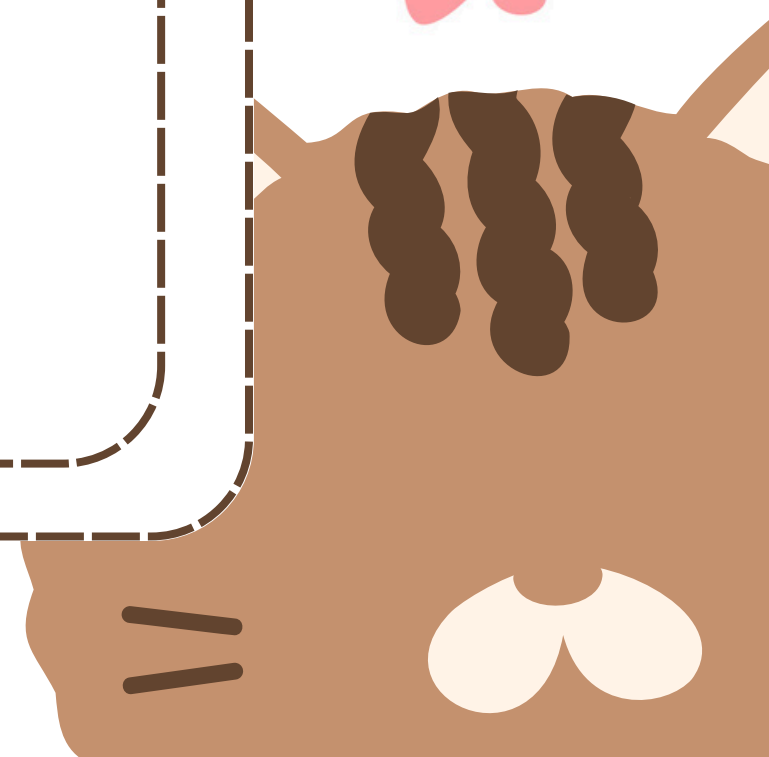


# 주요 변수 설명

- Type: 동물 종류 (1: 개, 2: 고양이)
- Age: 나이 (개월 단위)
- Breed1/Breed2: 주요 품종 / 부 품종 (숫자 ID)
- Gender: 성별 (1: 수컷, 2: 암컷, 3: 중성)
- Color1/Color2/Color3: 주요 색상 ID (최대 3개)
- MaturitySize: 체격 크기 (1~4 범주)
- FurLength: 털 길이
- Vaccinated/Dewormed/Sterilized: 예방접종/구충/중성화 여부 (1: 예, 2: 아니오, 3: 모름)



# 주요 변수 설명

- Health: 건강 상태 (1: 건강, 2: 경미한 질환, 3: 심각한 질환)
  - Quantity: 한 번에 등록된 마리 수
  - Fee: 입양 비용
  - PhotoAmt: 등록된 사진 수
  - VideoAmt: 등록된 동영상 수
  - Description: 설명 텍스트
  - RescuerID: 구조자 ID
  - PetID: 개별 고유 ID
- 

The background is white and decorated with various cat-themed elements. There are several paw prints in blue, pink, and yellow scattered around. At the bottom corners, there are stylized brown cat faces with white whiskers and bell-shaped noses. A dashed brown line forms a rectangular frame around the central text.

# EDA 및 데이터 전처리

PetFinder



# EDA

```
[ ] # Data Shape
```

```
print("Overall Dataset Shape: ",pet_data.shape)  
print("Dogs Dataset Shape: ",dogs_df.shape)  
print("Cats Dataset Shape: ",cats_df.shape)
```

```
⇒ Overall Dataset Shape: (14993, 24)  
Dogs Dataset Shape: (8132, 24)  
Cats Dataset Shape: (6861, 24)
```



# EDA

```
# Data Info
pet_data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14993 entries, 0 to 14992
Data columns (total 24 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Type             14993 non-null  int64
1   Name             13728 non-null  object
2   Age              14993 non-null  int64
3   Breed1           14993 non-null  int64
4   Breed2           14993 non-null  int64
5   Gender           14993 non-null  int64
6   Color1           14993 non-null  int64
7   Color2           14993 non-null  int64
8   Color3           14993 non-null  int64
9   MaturitySize     14993 non-null  int64
10  FurLength        14993 non-null  int64
11  Vaccinated       14993 non-null  int64
12  Dewormed         14993 non-null  int64
13  Sterilized       14993 non-null  int64
14  Health           14993 non-null  int64
15  Quantity         14993 non-null  int64
16  Fee              14993 non-null  int64
17  State            14993 non-null  int64
18  RescuerID        14993 non-null  object
19  VideoAmt         14993 non-null  int64
20  Description      14980 non-null  object
21  PetID            14993 non-null  object
22  PhotoAmt         14993 non-null  float64
23  AdoptionSpeed    14993 non-null  int64
dtypes: float64(1), int64(19), object(4)
memory usage: 2.7+ MB
```

```
[ ] pet_data.describe()

Type      Age      Breed1      Breed2      Gender      Color1      Color2      Color3
count  14993.000000  14993.000000  14993.000000  14993.000000  14993.000000  14993.000000  14993.000000
mean      1.457614    10.452078    265.272594    74.009738     1.776162     2.234176     3.222837
std       0.498217    18.155790     60.056818    123.011575     0.681592     1.745225     2.742562
min       1.000000     0.000000     0.000000     0.000000     1.000000     1.000000     0.000000
25%       1.000000     2.000000    265.000000     0.000000     1.000000     1.000000     0.000000
50%       1.000000     3.000000    266.000000     0.000000     2.000000     2.000000     0.000000
75%       2.000000    12.000000    307.000000    179.000000     3.000000     6.000000     5.000000
max       2.000000    255.000000    307.000000    307.000000     3.000000     7.000000     7.000000
```

# EDA

- 결측치 확인



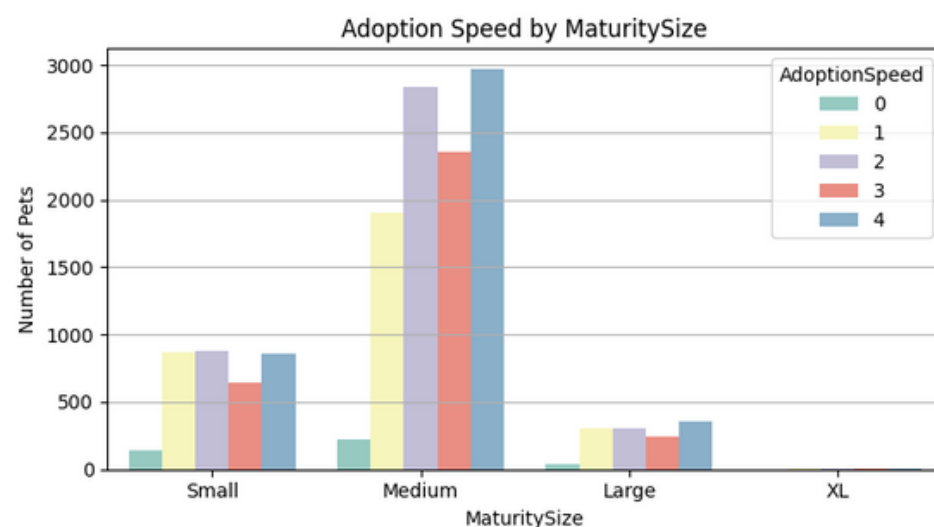
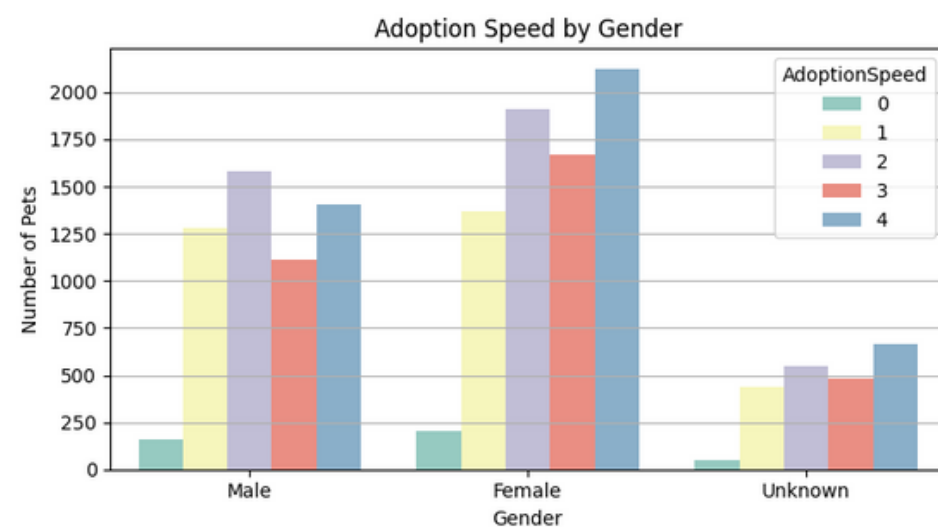
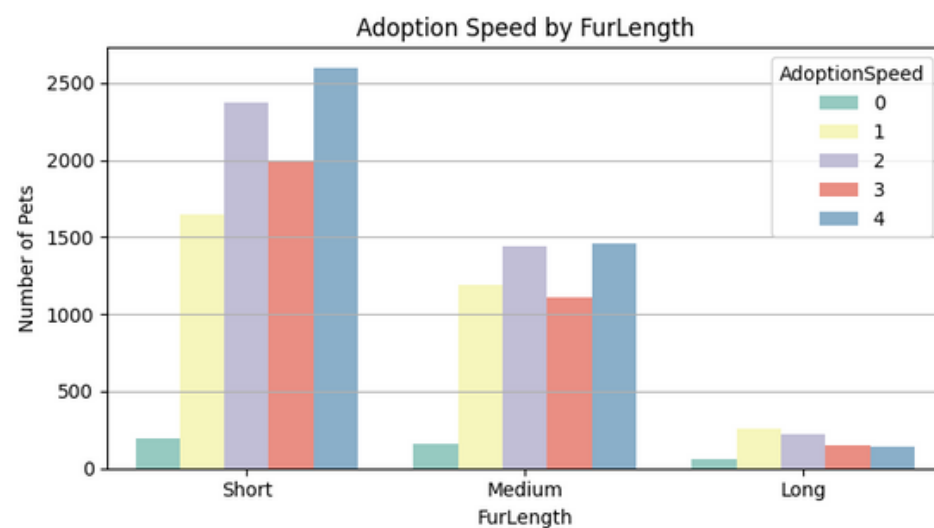
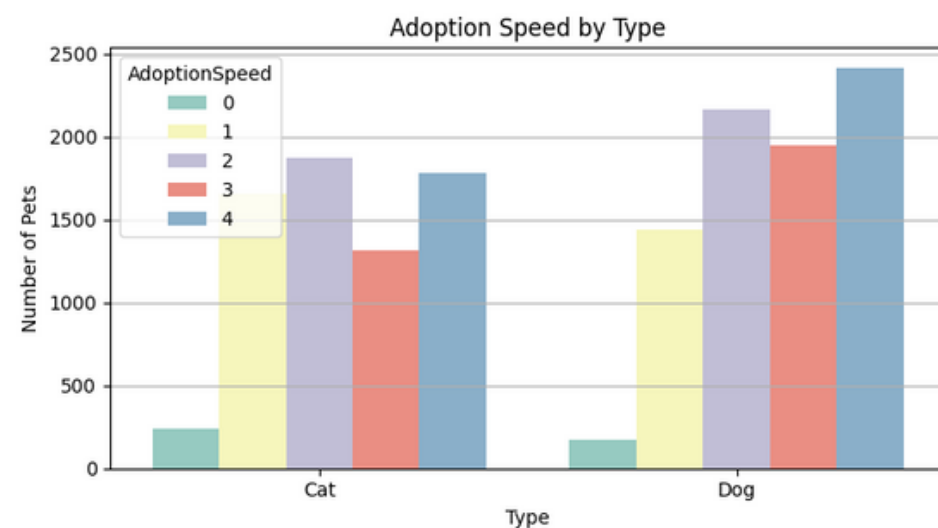
```
# Null : Name, Description  
pet_data.isnull().sum()
```

|      |      |
|------|------|
| Name | 1265 |
|------|------|

|             |    |
|-------------|----|
| Description | 13 |
|-------------|----|

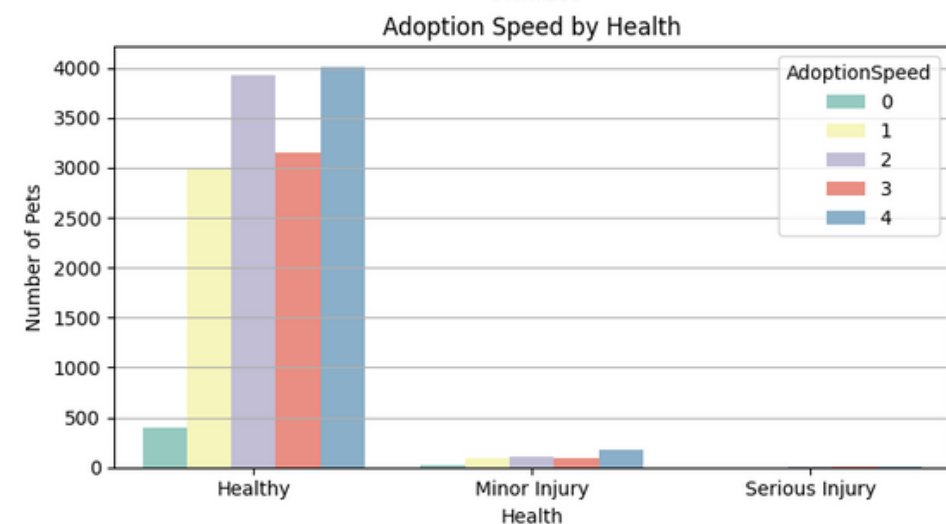
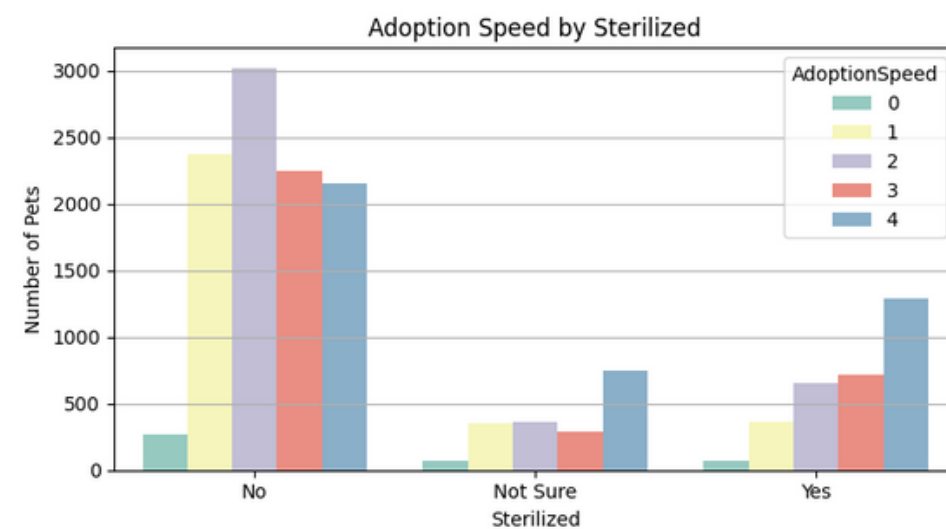
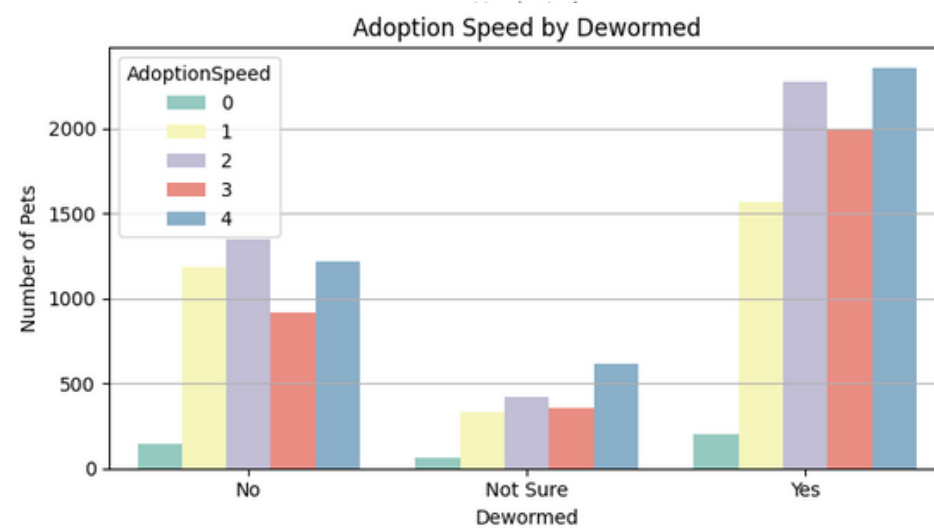
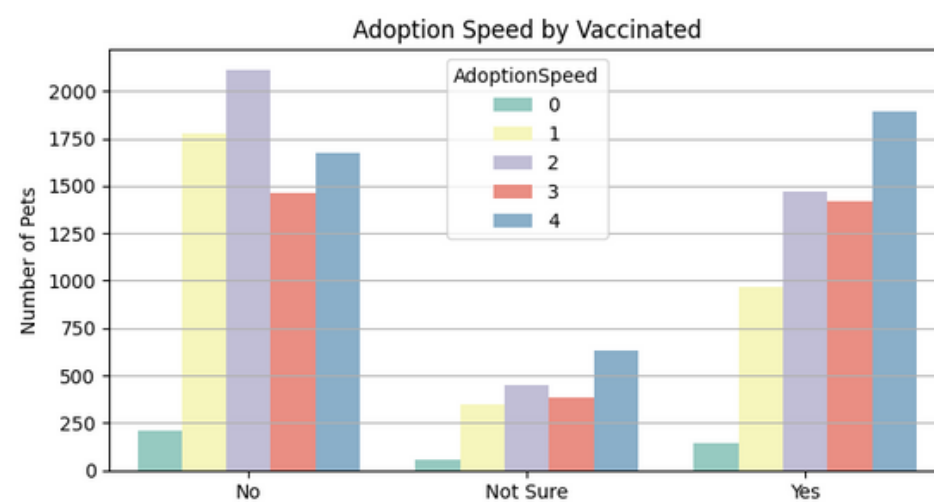
# EDA

- 범주형 변수 vs Adoption Speed



# EDA

- 범주형 변수 vs Adoption Speed





# EDA

- **범주형 변수** vs Adoption Speed

Type: Dog가 Cat보다 전반적으로 입양이 빠름

Gender : Female일 때 빠르게 입양

Maturity Size : Medium이 압도적으로 많으며 입양 속도도 가장 높음

Vaccinated : Yes일 때 AdoptionSpeed=4 비율이 높고 전반적으로 빠  
름

Health : Healthy가 거의 대부분이며 입양속도도 양호

Breed : Dog는 Mixed Breed가 압도적으로 가장 많음 나머지 인기 품종  
은 Labrador Retriever, Shih Tzu, Poodle, Terrier, Golden  
Retriever 등. Cat는 Domestic Short Hair가 가장 많음

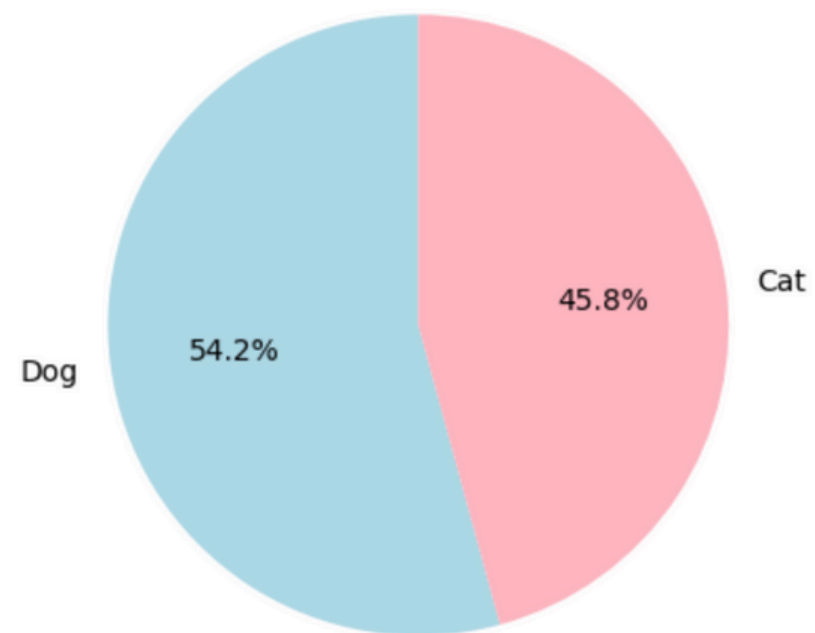




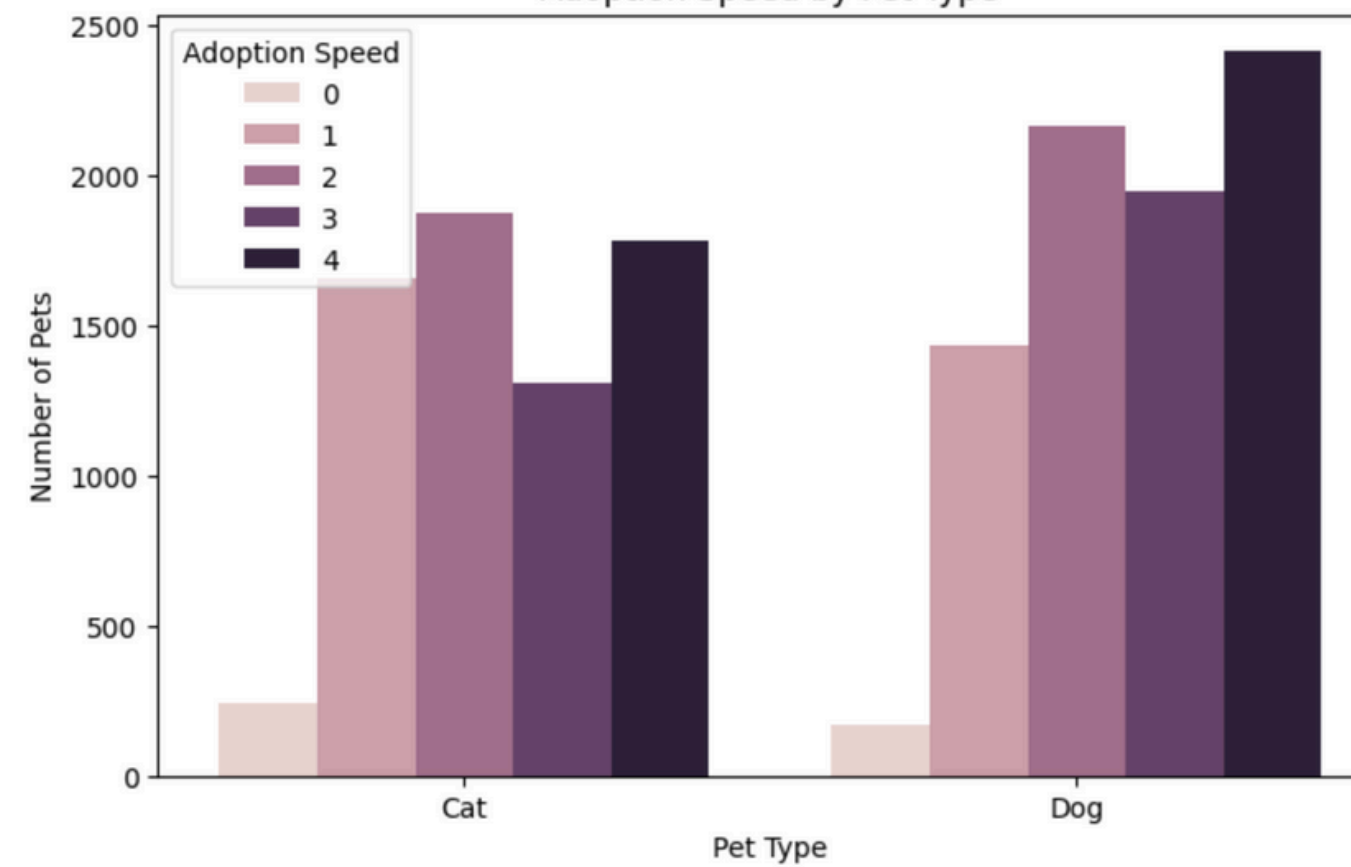
# EDA

- Type

Distribution of Pet Types



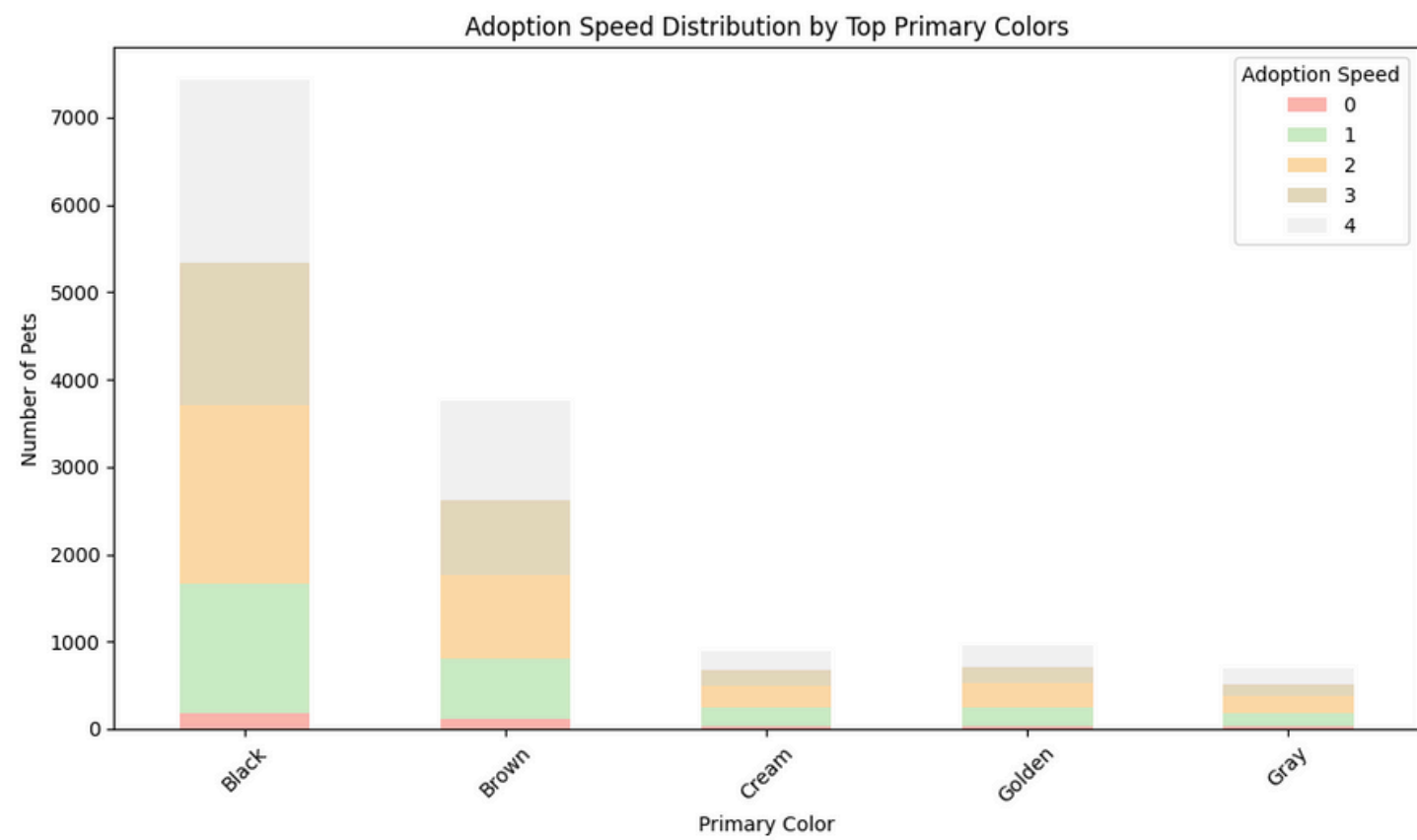
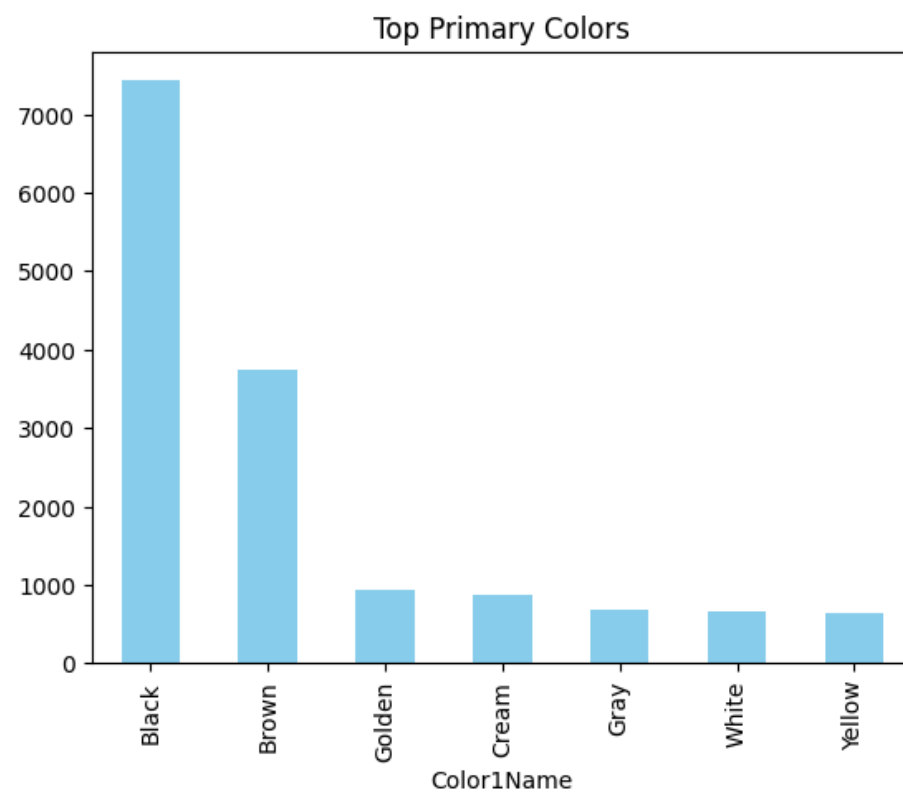
Adoption Speed by Pet Type





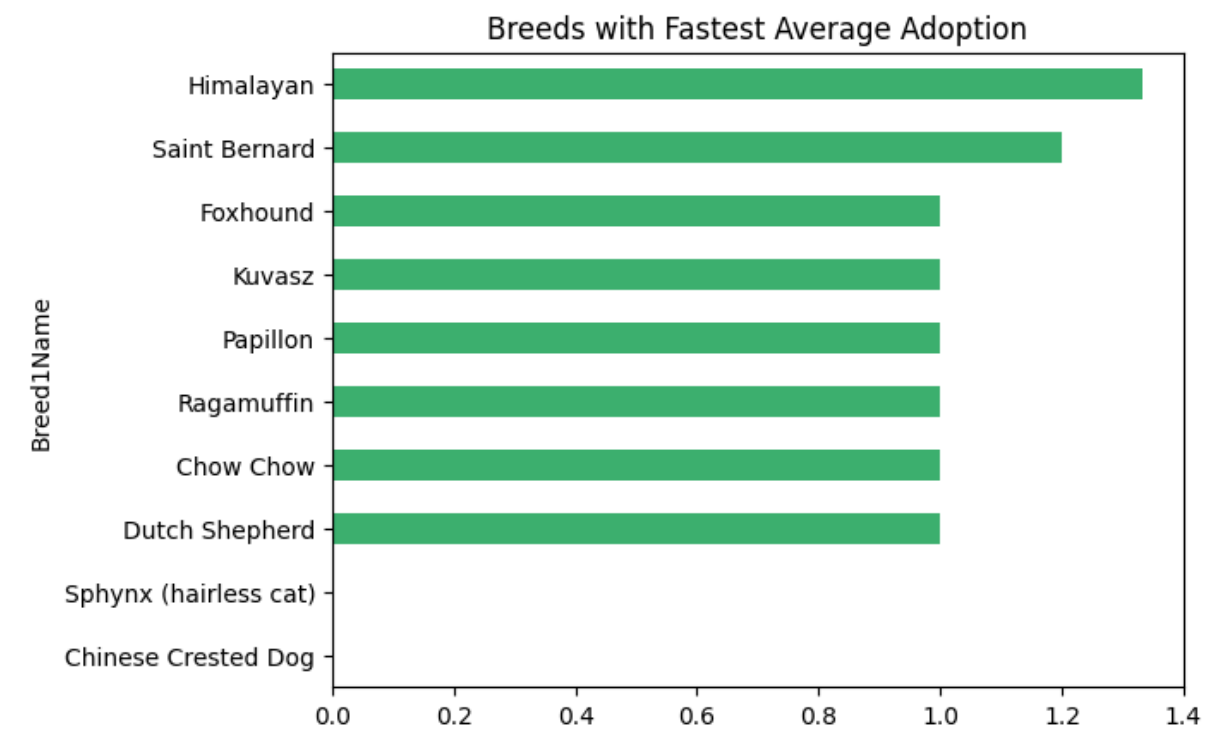
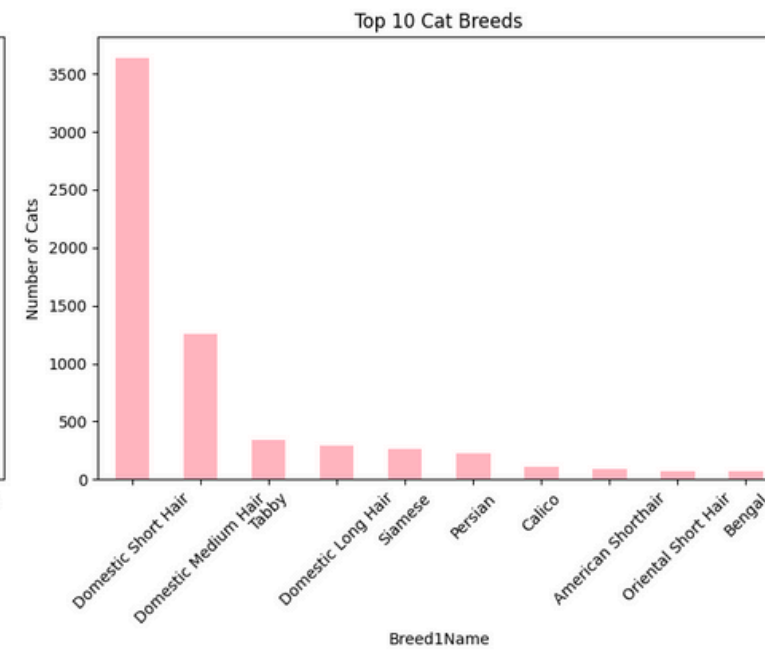
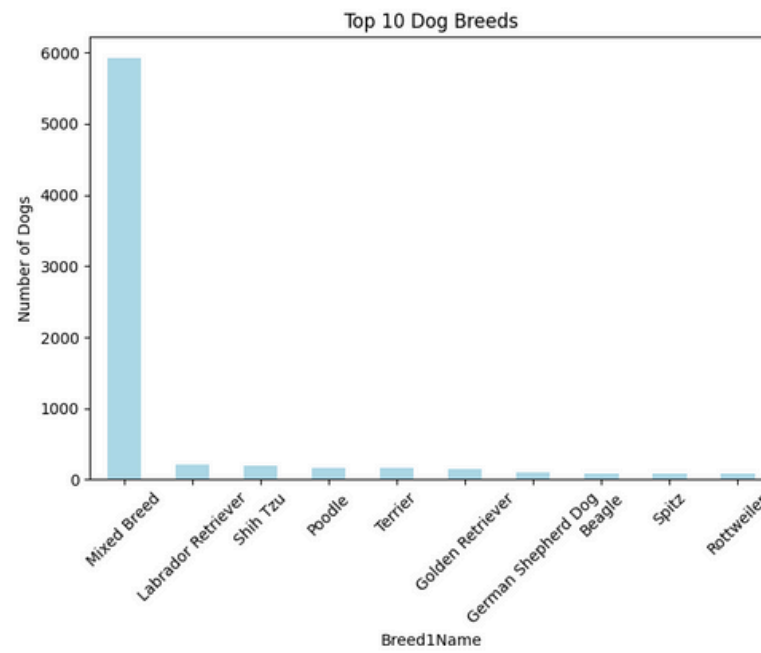
# EDA

- Color



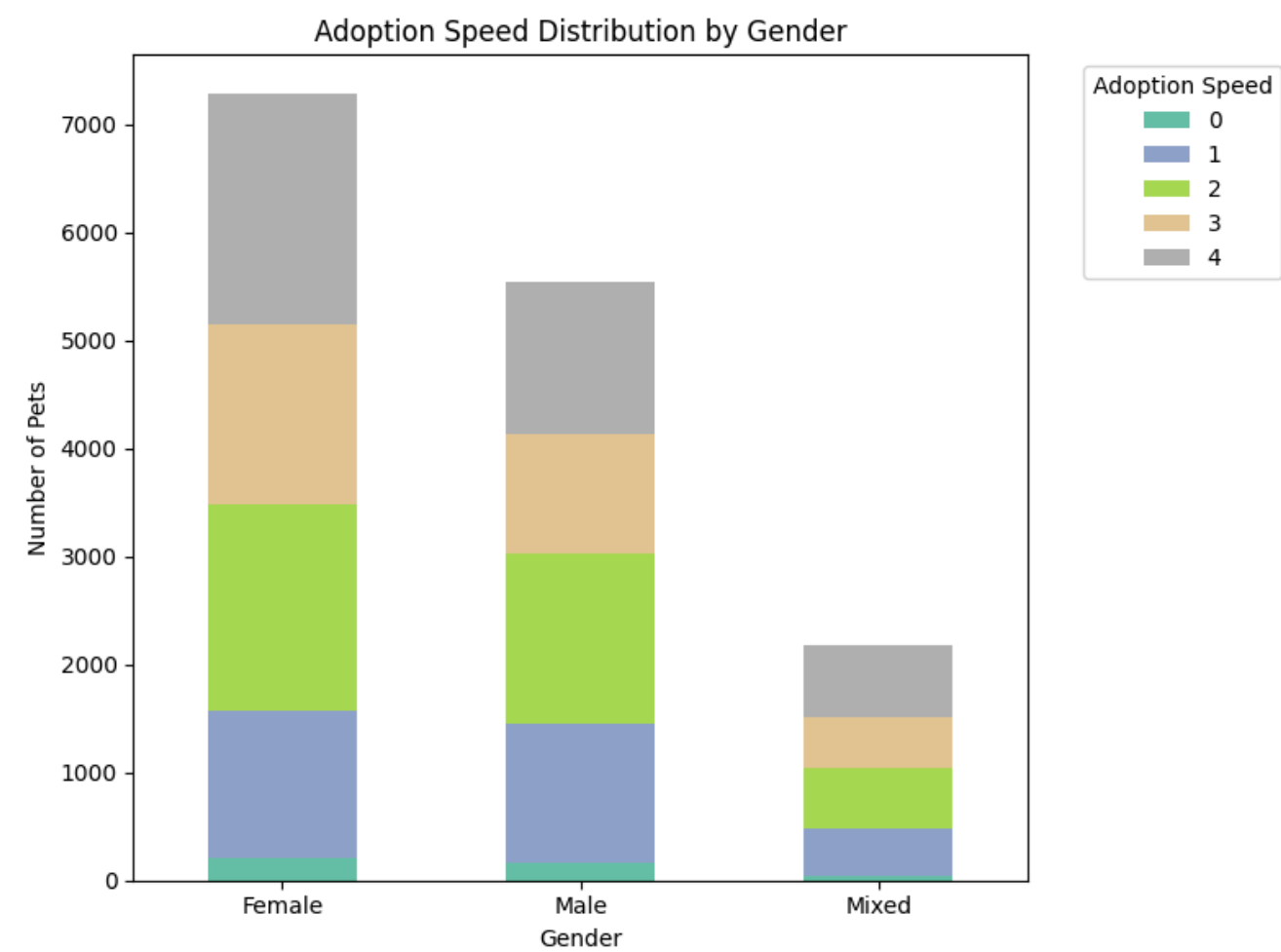
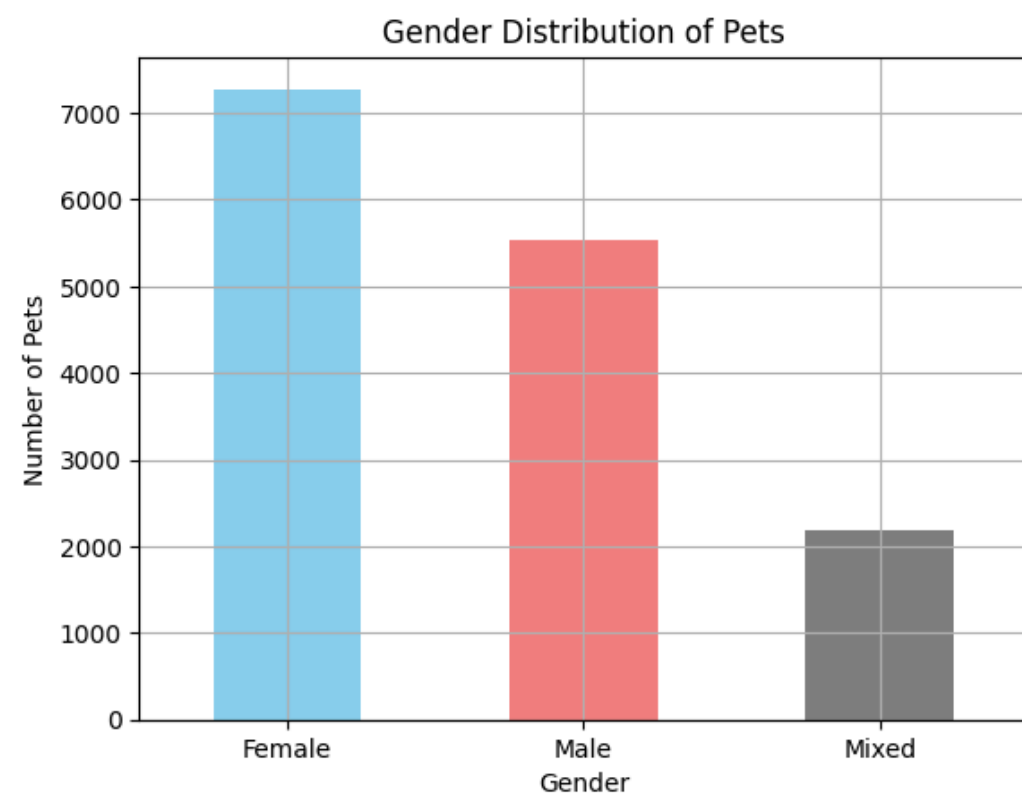
# EDA

- Breed



# EDA

- Gender





# EDA

- 수치형 변수 vs Adoption Speed

Age

- 어릴수록 입양 가능성 높음
- 2~5yr은 입양이 느림 → Speed 4가 증가

Fee

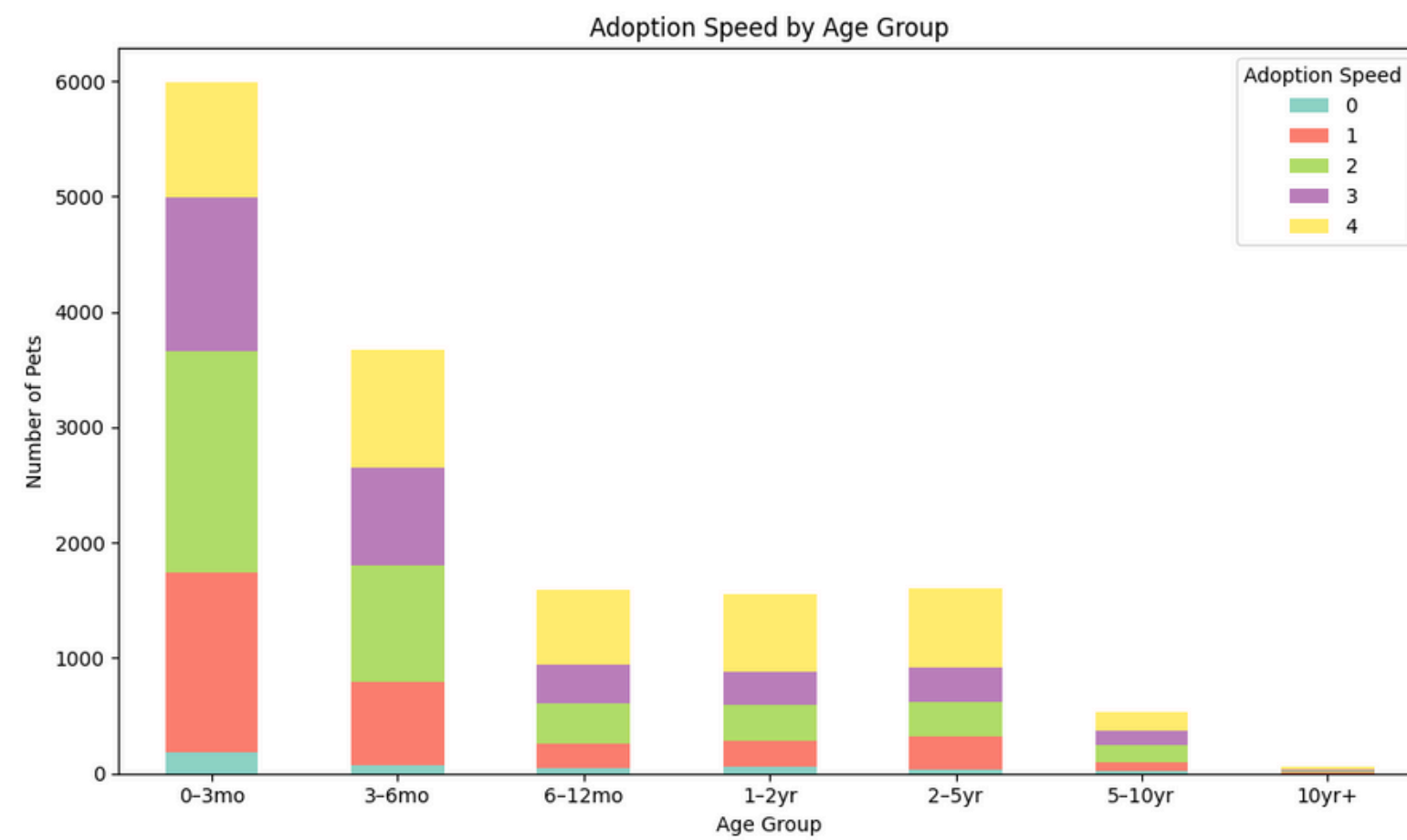
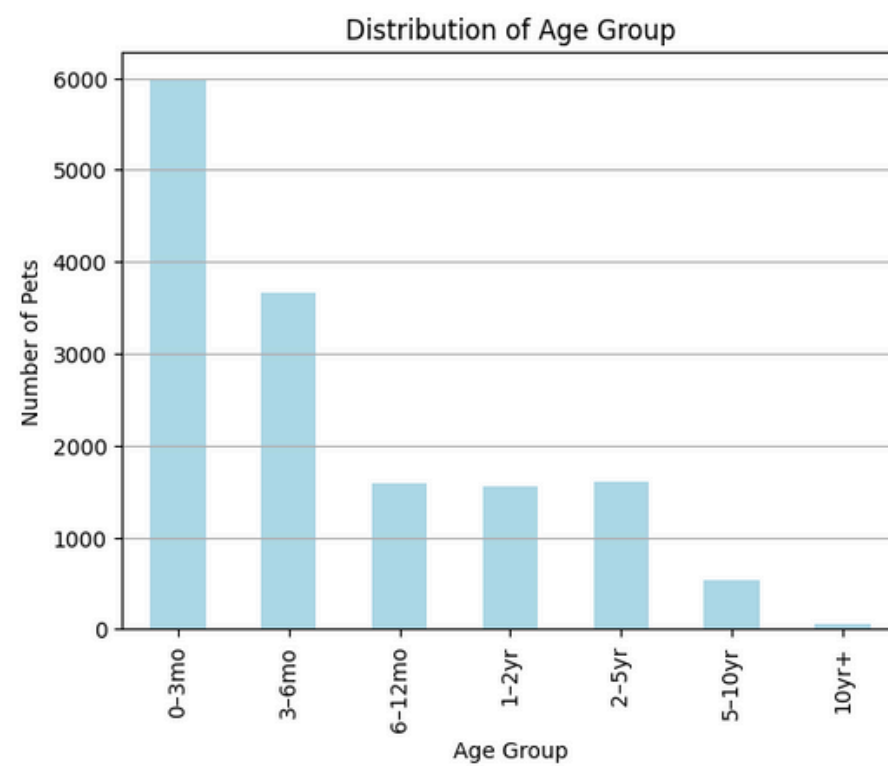
- 대부분의 동물은 Fee가 낮음 또는 0원
- 일부 높은 입양비를 가진 경우(outlier)는 전반적으로 입양이 느린 편

PhotoAmt

- 평균적으로 AdoptionSpeed=0, 1쪽에서 사진 수가 더 많음

# EDA

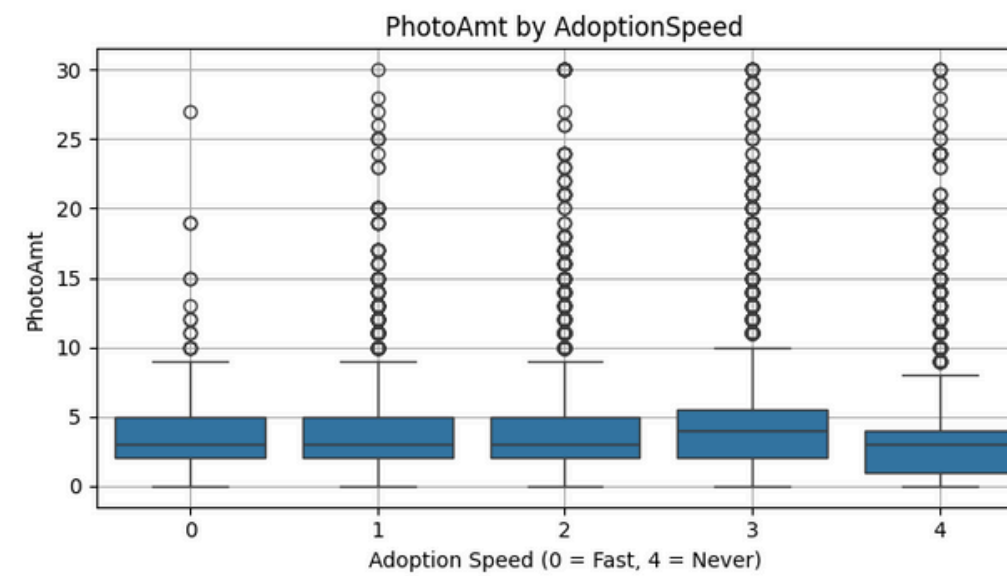
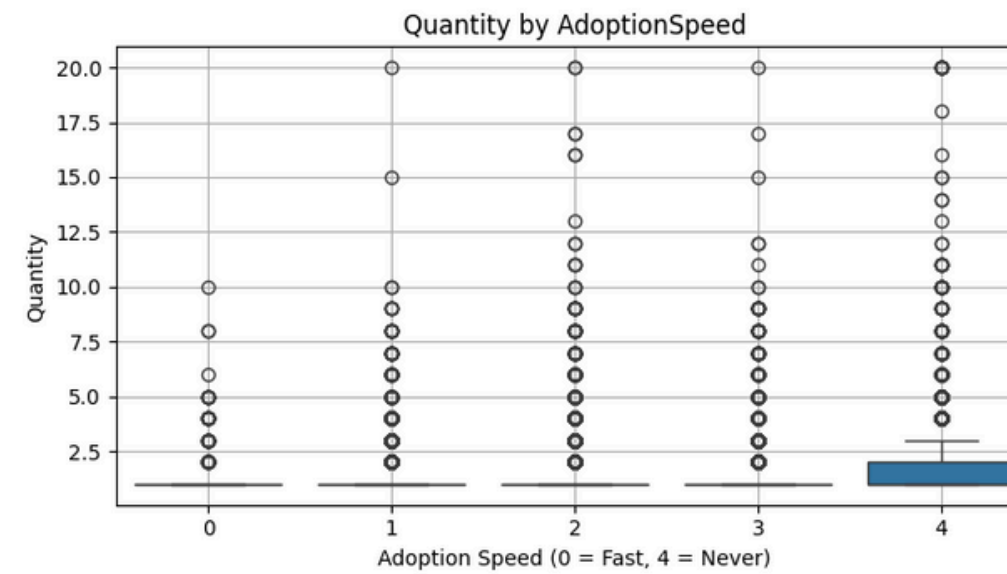
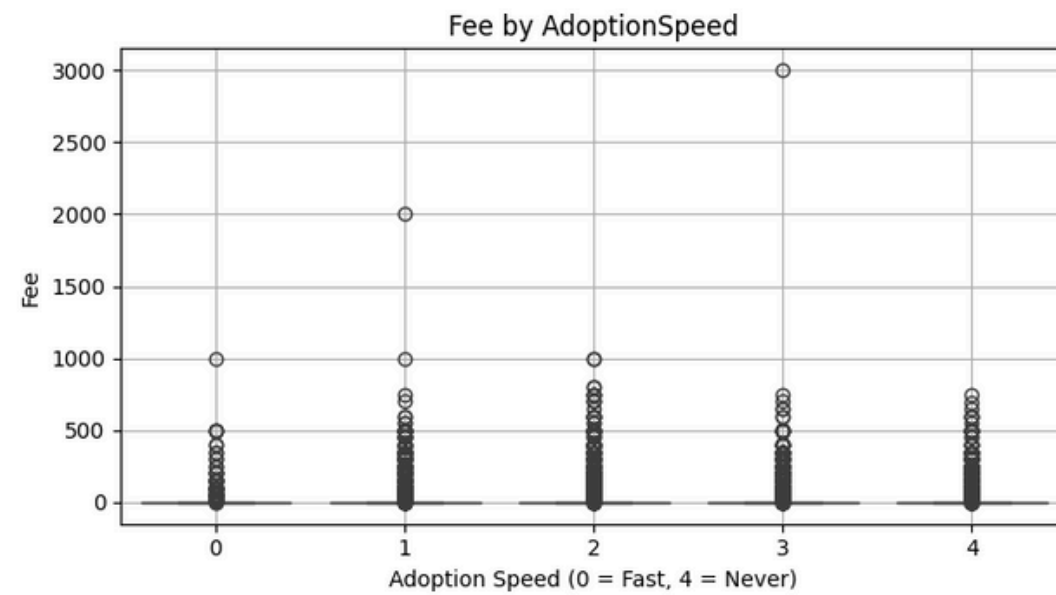
- Age





# EDA

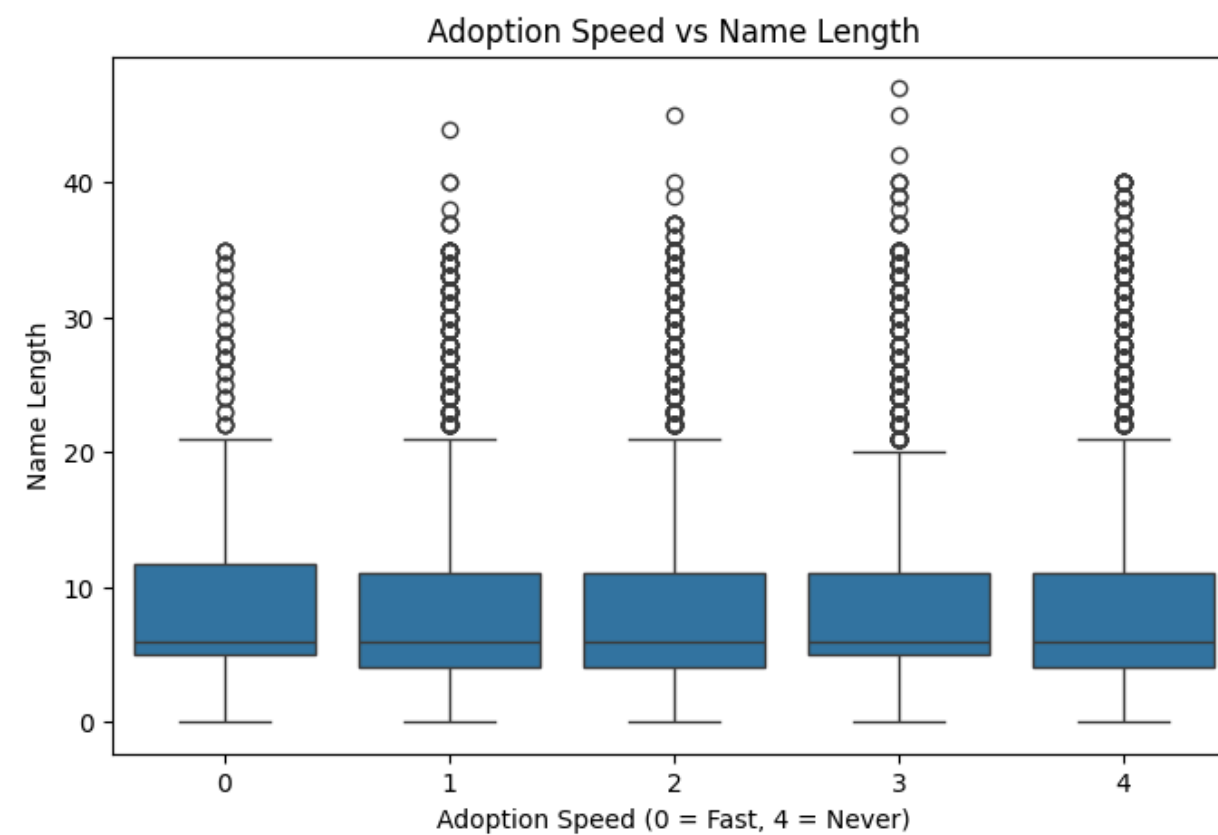
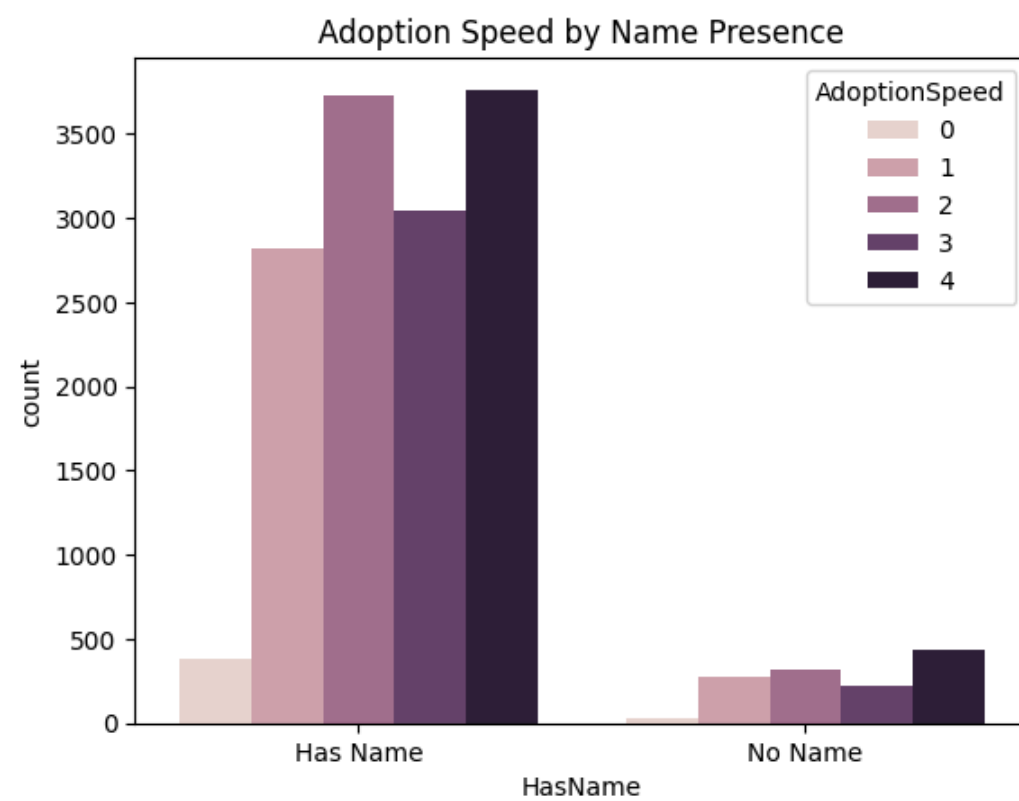
- Others





# EDA

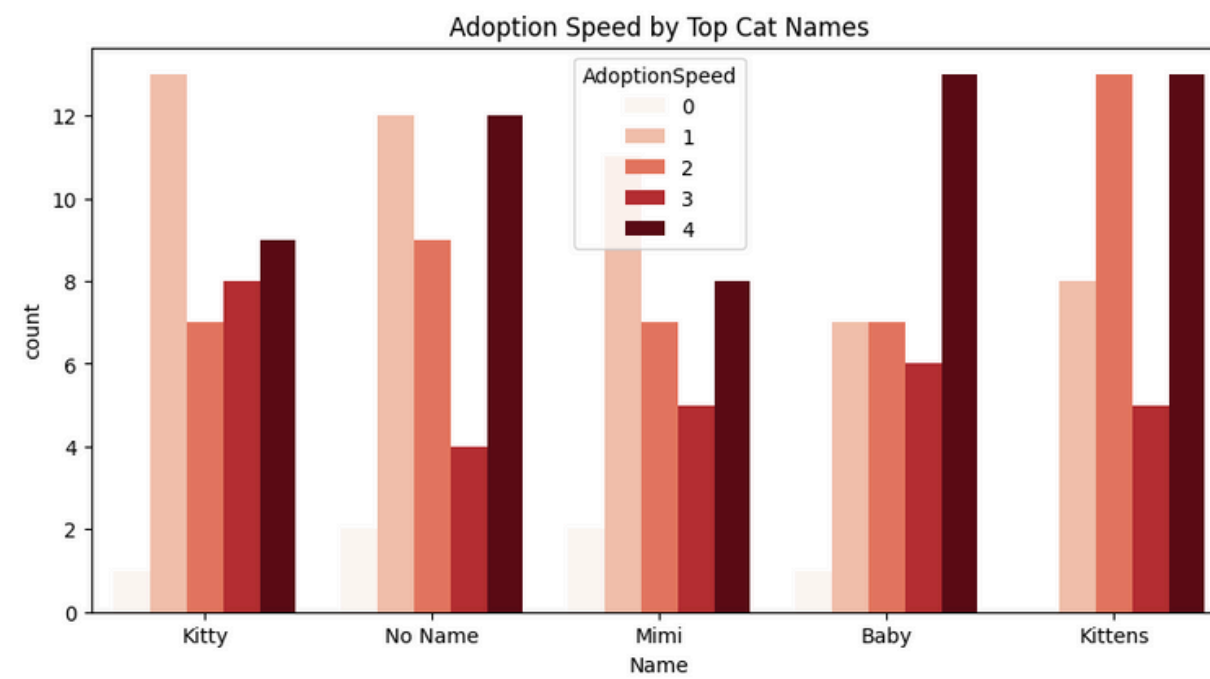
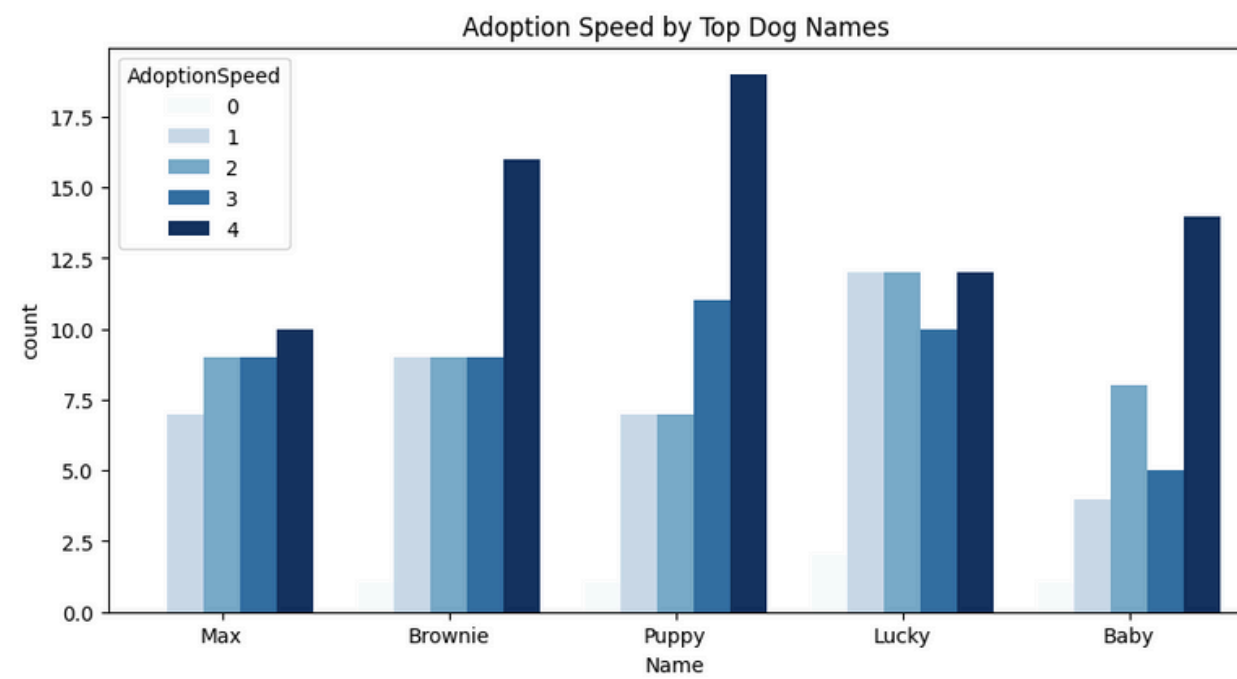
- Name

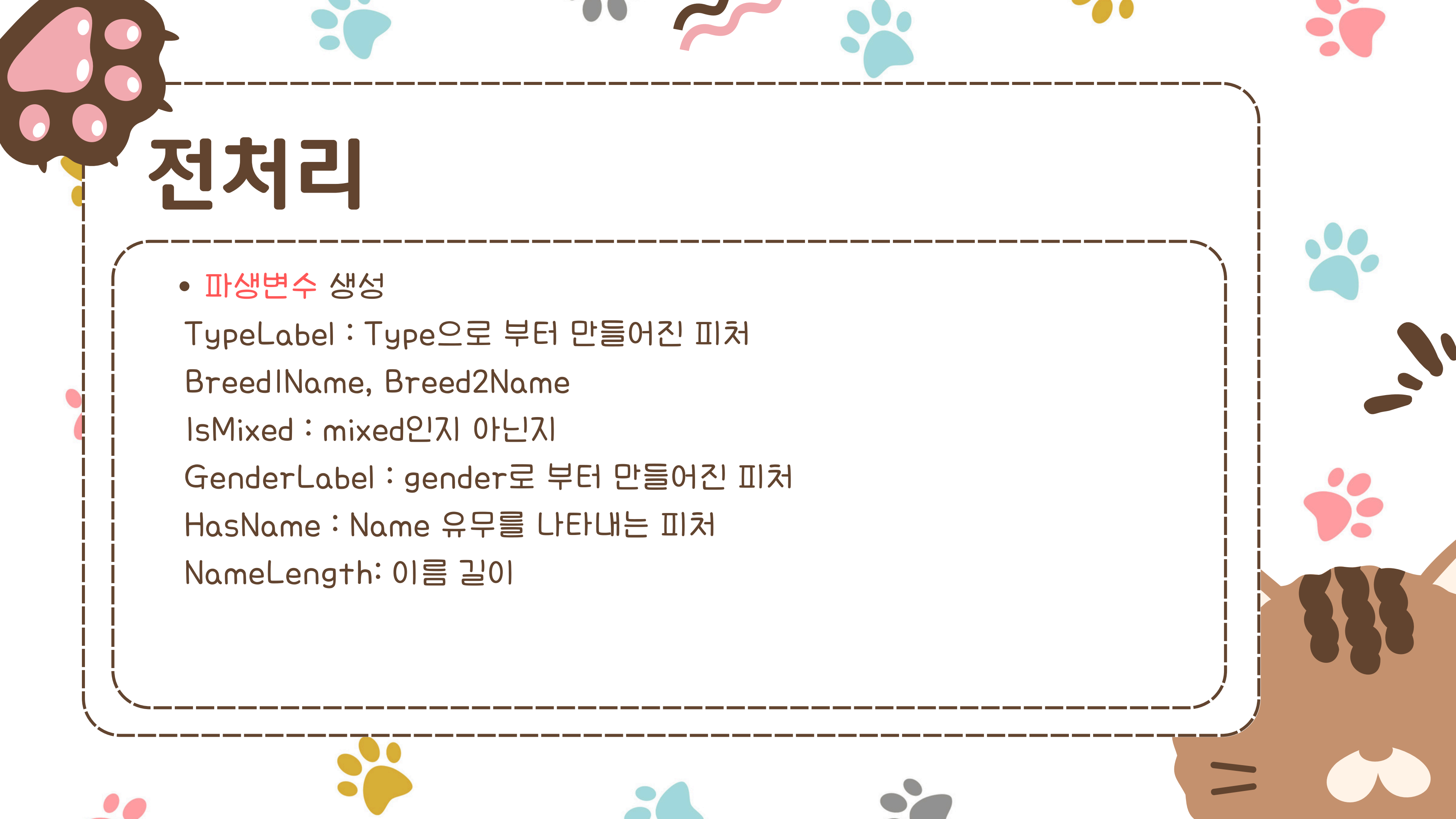


Has Name 파생변수 생성!

# EDA

- Name





# 전처리

- 파생변수 생성

TypeLabel : Type으로 부터 만들어진 피쳐

Breed1Name, Breed2Name

IsMixed : mixed인지 아닌지

GenderLabel : gender로 부터 만들어진 피쳐

HasName : Name 유무를 나타내는 피쳐

NameLength: 이름 길이

# 전처리

- 범주형 변수 One Hot Encoding

```
[ ] # One Hot Encoding
import pandas as pd

categorical_cols = ['Type', 'Gender', 'MaturitySize', 'FurLength',
                   'Vaccinated', 'Dewormed', 'Sterilized', 'Health']

pet_data_encoded = pd.get_dummies(pet_data, columns=categorical_cols, drop_first=True)

print("New shape after encoding:", pet_data_encoded.shape)
pet_data_encoded.columns
```

```
⇒ New shape after encoding: (14993, 32)
Index(['Name', 'Age', 'Breed1', 'Breed2', 'Color1', 'Color2', 'Color3',
      'Quantity', 'Fee', 'State', 'RescuerID', 'VideoAmt', 'Description',
      'PetID', 'PhotoAmt', 'AdoptionSpeed', 'Type_2', 'Gender_2', 'Gender_3',
      'MaturitySize_2', 'MaturitySize_3', 'MaturitySize_4', 'FurLength_2',
      'FurLength_3', 'Vaccinated_2', 'Vaccinated_3', 'Dewormed_2',
      'Dewormed_3', 'Sterilized_2', 'Sterilized_3', 'Health_2', 'Health_3'],
      dtype='object')
```

# 전처리

- 수치형 변수 Scaling

```
[ ] import joblib
    from sklearn.preprocessing import StandardScaler

    numerical_cols = ['Type', 'Age', 'Fee', 'DescLengthWord', 'NameLength', 'Gender',
                     'MaturitySize', 'FurLength', 'Vaccinated', 'Quantity', 'PhotoAmt']

    scaler = StandardScaler()
    scaled_num = scaler.fit_transform(pet_data_new[numerical_cols])
    scaled_num_df = pd.DataFrame(scaled_num, columns=numerical_cols, index=pet_data_new.index)

    print(scaled_num_df.head(3))
    joblib.dump(scaler, 'scaler.pkl')
```

```
⇒
```

|   | Type      | Age       | Fee       | DescLengthWord | NameLength | Gender    | \ |
|---|-----------|-----------|-----------|----------------|------------|-----------|---|
| 0 | 1.088692  | -0.410466 | 1.004184  | 0.103969       | -0.365056  | -1.138787 |   |
| 1 | 1.088692  | -0.520627 | -0.271132 | -0.572914      | 0.308115   | -1.138787 |   |
| 2 | -0.918534 | -0.520627 | -0.271132 | 0.103969       | -0.365056  | -1.138787 |   |

|   | MaturitySize | FurLength | Vaccinated | Quantity  | PhotoAmt  |
|---|--------------|-----------|------------|-----------|-----------|
| 0 | -1.573167    | -0.780377 | 0.402608   | -0.391237 | -0.828403 |
| 1 | 0.251848     | 0.888933  | 1.900450   | -0.391237 | -0.541680 |
| 2 | 0.251848     | 0.888933  | -1.095234  | -0.391237 | 0.891932  |

```
['scaler.pkl']
```



The background is white and decorated with various cat-themed elements. There are several paw prints in blue, pink, and yellow scattered around. At the bottom corners, there are stylized brown cat faces with white whiskers and bell-shaped noses. A dashed brown line forms a rectangular frame around the central text.

# Modeling

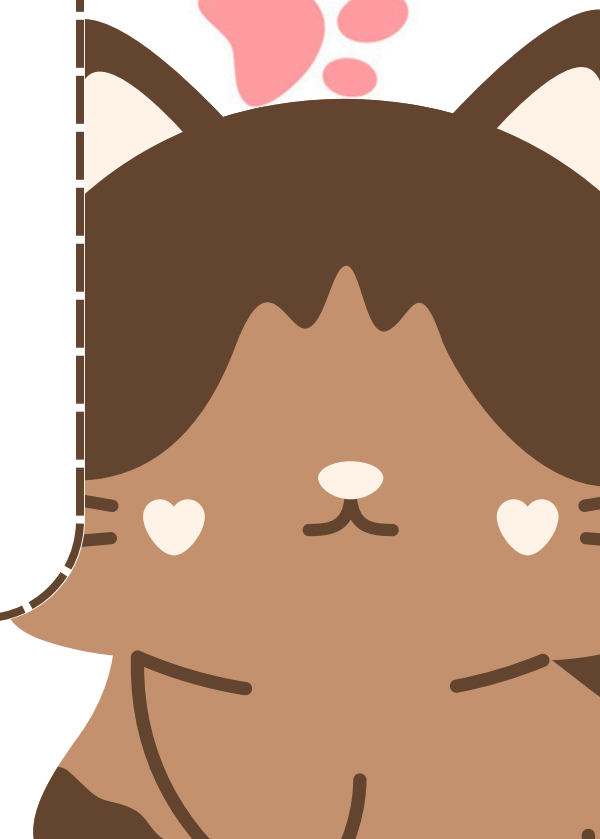
PetFinder



# Modeling

## [Overall Process]

1. 기본 모델 학습 및 평가
2. 하이퍼파라미터 튜닝
3. 최적 모델 평가
4. 중요 피처 시각화



# Modeling

- 1) Logistic Regression
- 2) Random Forest
- 3) LightGBM
- 4) XGBoost
- 5) Ensemble Methods



# Modeling

## I. Logistic Regression

# 2. 기본 모델 학습 및 평가

```
base_lr = LogisticRegression(max_iter=1000, random_state=42)
base_lr.fit(X_train, y_train)
y_pred = base_lr.predict(X_val)
print("-- Base Logistic Regression Performance --")
print("Accuracy:", accuracy_score(y_val, y_pred))
print("Weighted F1:", f1_score(y_val, y_pred, average='weighted'))
print("QWK:", cohen_kappa_score(y_val, y_pred, weights='quadratic'))
print("Confusion Matrix:\n", confusion_matrix(y_val, y_pred))
print(classification_report(y_val, y_pred))
```



# Modeling

## I. Logistic Regression

```
# 3. 하이퍼파라미터 튜닝
param_grid = {
    'C': [0.01, 0.1, 1, 10],
    'penalty': ['l2'],
    'solver': ['lbfgs', 'liblinear'],
    'multi_class': ['auto', 'multinomial']
}
grid = GridSearchCV(
    estimator=LogisticRegression(max_iter=1000, random_state=42),
    param_grid=param_grid,
    cv=3,
    scoring='accuracy',
    verbose=1,
    n_jobs=-1
)
grid.fit(X_train, y_train)
print("-- Grid Search Results --")
print("Best Parameters:", grid.best_params_)
print("Best CV Accuracy:", grid.best_score_)
```

# Modeling

## I. Logistic Regression

```
# 4. 최적 모델 평가
best_lr = grid.best_estimator_
y_best_pred = best_lr.predict(X_val)
print("-- Best Logistic Regression Performance --")
print("Accuracy:", accuracy_score(y_val, y_best_pred))
print("Weighted F1:", f1_score(y_val, y_best_pred, average='weighted'))
print("QWK:", cohen_kappa_score(y_val, y_best_pred, weights='quadratic'))
print("Confusion Matrix:\n", confusion_matrix(y_val, y_best_pred))
print(classification_report(y_val, y_best_pred))
```

# Modeling

## I. Logistic Regression

```
# 5. 중요 피쳐 시각화
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

coef = best_lr.coef_                                # shape = (n_classes, n_features)
imp = np.mean(np.abs(coef), axis=0)                  # feature 당 mean |coef|

# 시각화 예시: 합산 기준 Top 10
top10 = category_sum.head(10)

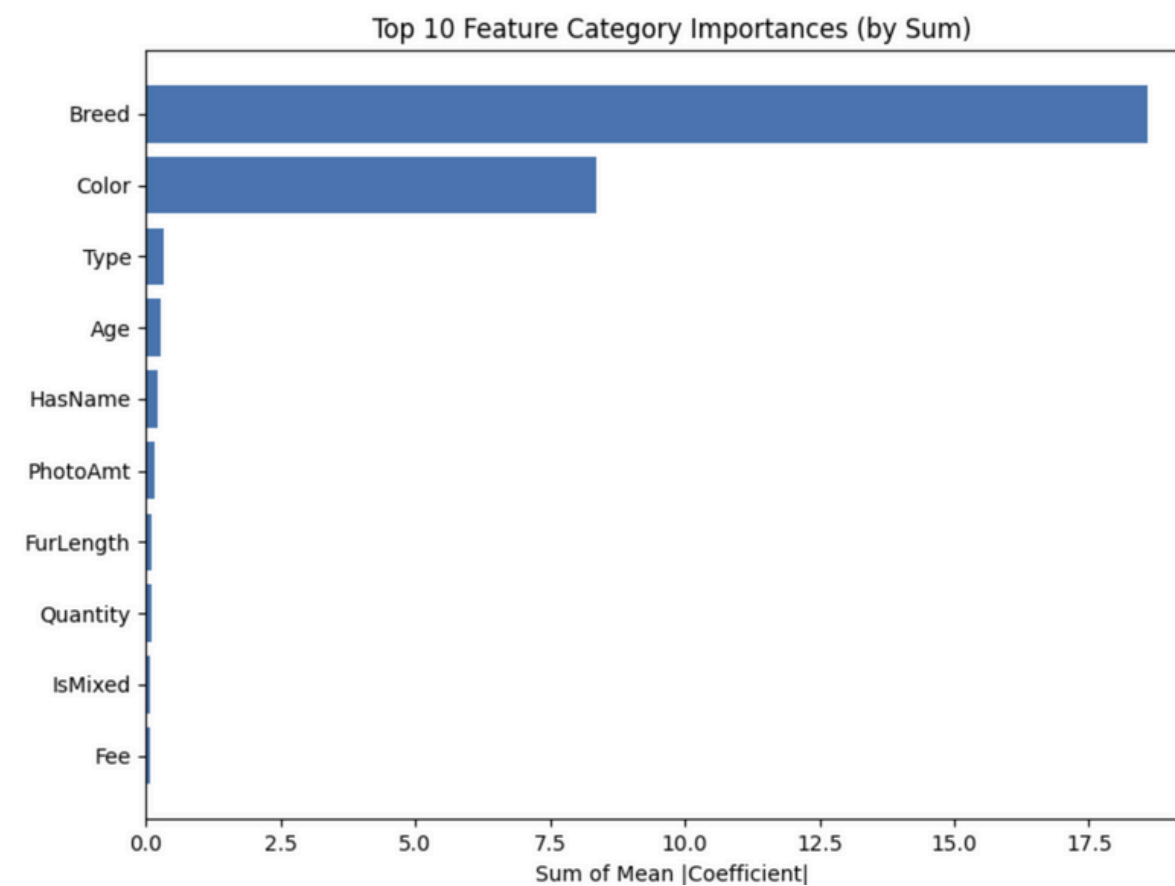
plt.figure(figsize=(8, 6))
plt.barh(top10[:-1].index, top10[:-1].values)
plt.xlabel('Sum of Mean |Coefficient|')
plt.title('Top 10 Feature Category Importances (by Sum)')
plt.tight_layout()
plt.show()
```



# Modeling

## I. Logistic Regression

| 기본 Accuracy | 최종 Accuracy | 상위 10개 중요 변수                                                                                    |
|-------------|-------------|-------------------------------------------------------------------------------------------------|
| 0.34678     | 0.34744     | Breed<br>Color<br>Type<br>Age<br>HasName<br>PhotoAmt<br>FurLength<br>Quantity<br>IsMixed<br>Fee |



# Modeling

## 2. Random Forest

# 1. 모델 정의

```
rf = RandomForestClassifier(random_state=42)
```

# 2. 하이퍼파라미터 그리드 정의

```
param_grid = {  
    'n_estimators': [100, 200],  
    'max_depth': [None, 10, 20],  
    'min_samples_split': [2, 5],  
    'min_samples_leaf': [1, 2],  
    'max_features': ['sqrt', 'log2']  
}
```

# 3. GridSearchCV

```
grid_search = GridSearchCV(  
    estimator=rf,  
    param_grid=param_grid,  
    cv=3,  
    n_jobs=-1,  
    verbose=2,  
    scoring='accuracy'  
)
```

# 4. 학습 시작

```
grid_search.fit(X_train, y_train)
```

# 3-Fold 교차 검증  
# 병렬처리  
# 출력 레벨  
# 평가 기준

# Modeling

## 2. Random Forest

```
# 5. 최적 파라미터 및 점수 출력
print("\n Best Parameters:")
print(grid_search.best_params_)

print(f"\nBest Cross-Validation Accuracy: {grid_search.best_score_:.4f}")

# 6. 최적 모델 기반 예측
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)

# 7. 성능 평가
print(f"Test Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print(classification_report(y_test, y_pred))
```

# Modeling

## 2. Random Forest

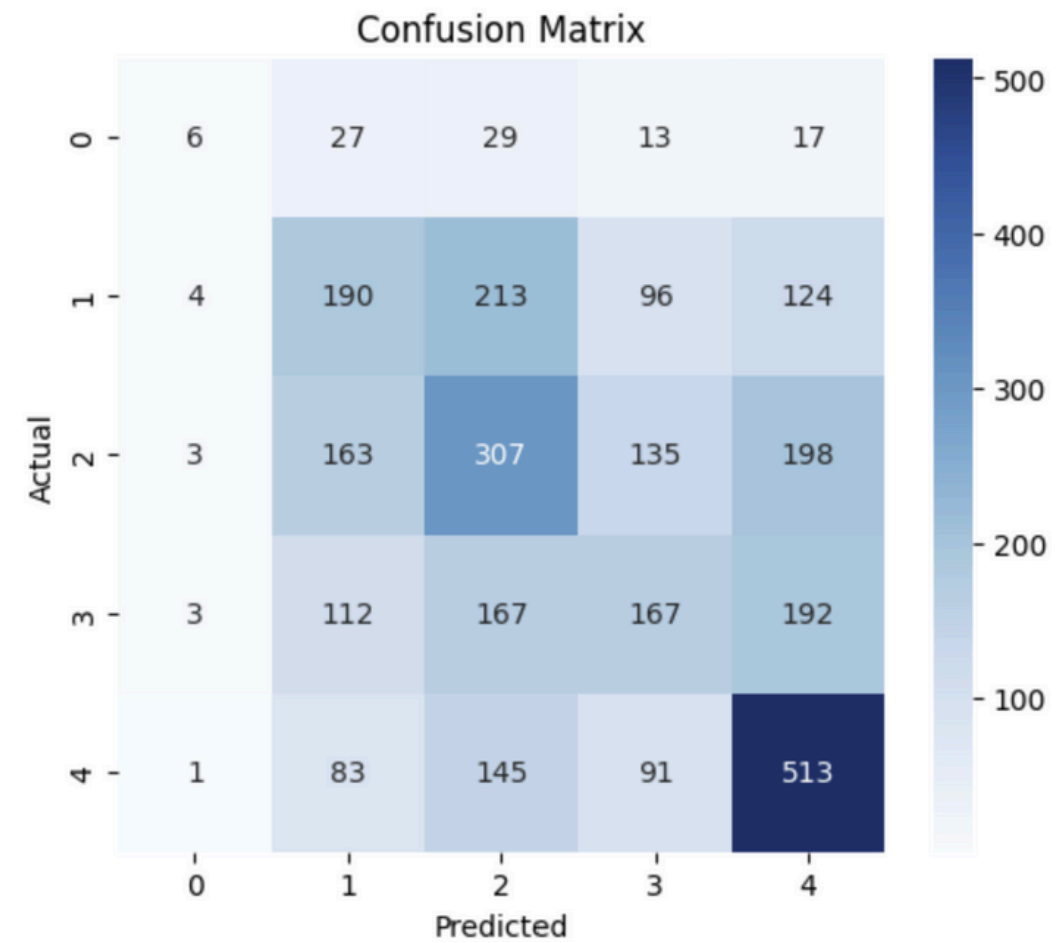
```
# 혼동 행렬 계산
cm = confusion_matrix(y_test, y_pred)

# 시각화
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=rf_model.classes_, ytickla
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```

# Modeling

## 2. Random Forest

| 기본 Accuracy | 최종 Accuracy | 상위 10개 중요 변수                                                                                                     |
|-------------|-------------|------------------------------------------------------------------------------------------------------------------|
| 0.4215      | 0.42        | DescLengthWord<br>NameLength<br>Age<br>PhotoAmt<br>Vaccinated<br>Gender<br>Quantity<br>FurLength<br>MaturitySize |





# Modeling

## 3. LightGBM

```
# SMOTE와 모델을 포함한 파이프라인
pipeline = Pipeline([
    ('smote', SMOTE(random_state=42)),
    ('model', LGBMClassifier(random_state=42))
])

# parameter grid
param_grid = {
    'model__n_estimators': [100, 200],
    'model__max_depth': [5, 10, -1],
    'model__learning_rate': [0.05, 0.1]
}
```

```
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    scoring='f1_weighted', # 멀티클래스라면 f1_weighted 또는 accuracy 사용
    cv=3,
    n_jobs=-1,
    verbose=1
)

# 학습
grid_search.fit(X_train, y_train)

best_model = grid_search.best_estimator_

# 예측
y_pred_best = best_model.predict(X_test)
```

# Modeling

## 3. LightGBM

```
# 최적 파라미터
print("Best params:", study.best_params)

importances = best_model.named_steps['model'].feature_importances_
feature_names = X_train.columns
importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
importance_df.sort_values(by='Importance', ascending=False).head(10)

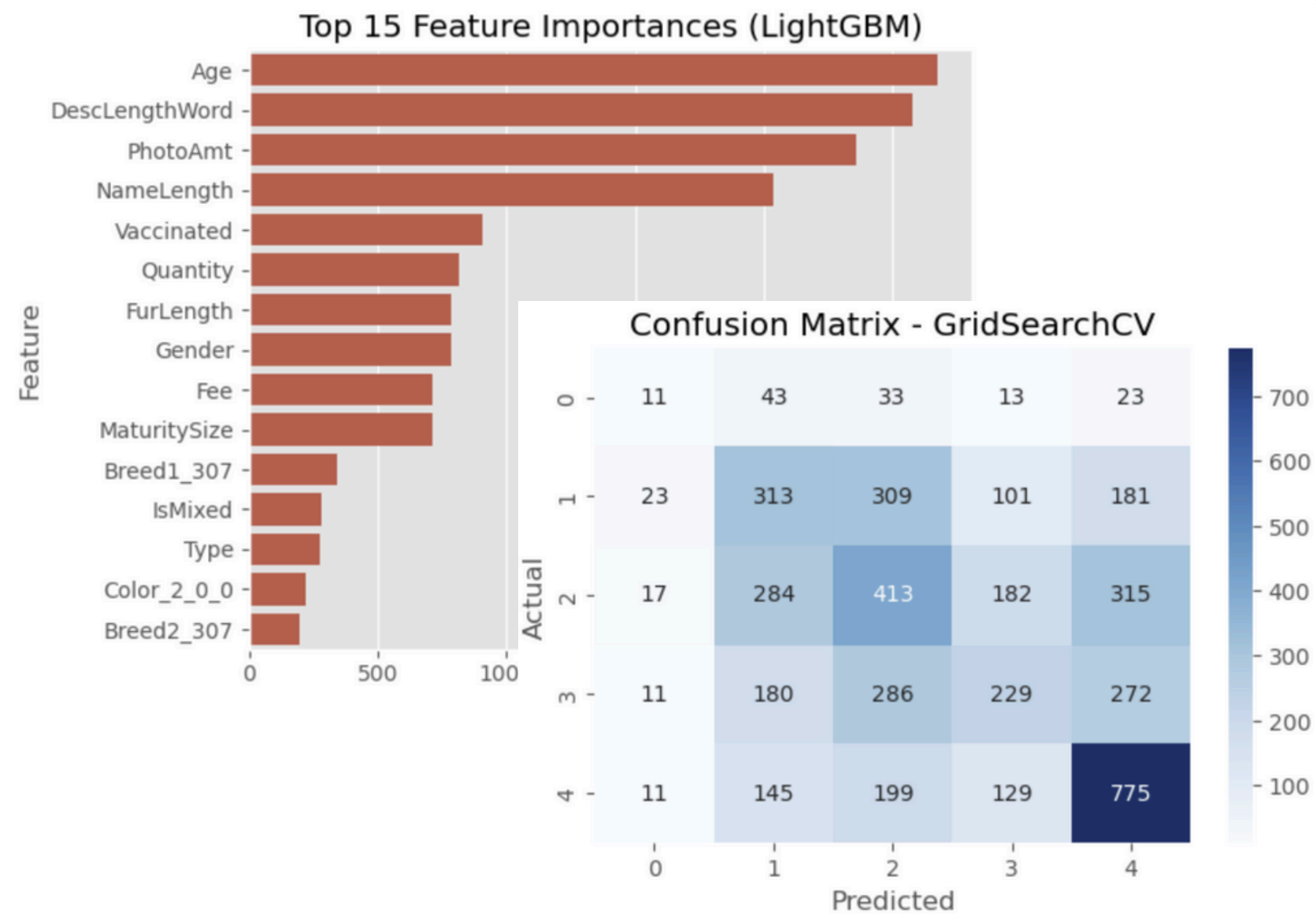
sns.barplot(data=importance_df.sort_values('Importance', ascending=False).head(15),
            x='Importance', y='Feature')
plt.title('Top 15 Feature Importances (LightGBM)')
plt.tight_layout()
plt.show()

plt.figure(figsize=(6, 4))
sns.heatmap(confusion_matrix(y_test, y_pred_final), annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix - GridSearchCV')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```

# Modeling

## 3. LightGBM

| 기본 Accuracy | 최종 Accuracy | 상위 10개 중요 변수                                                                                                            |
|-------------|-------------|-------------------------------------------------------------------------------------------------------------------------|
| 0.39        | 0.3766      | DescLengthWord<br>Age<br>NameLength<br>PhotoAmt<br>Vaccinated<br>Fee<br>Gender<br>Quantity<br>FurLength<br>MaturitySize |



# Modeling

## 4. XGBoost

```
# 모델 정의 및 학습
xgb = XGBClassifier(
    objective='multi:softprob',
    num_class=5,
    eval_metric='mlogloss',
    use_label_encoder=False,
    random_state=42
)

xgb.fit(X_train, y_train)

# 예측 및 평가
y_pred = xgb.predict(X_val)
print(confusion_matrix(y_val, y_pred))
print(classification_report(y_val, y_pred))
```

# Modeling

## 4. XGBoost

```
# 하이퍼파라미터 후보 정의
param_grid = {
    'max_depth': [4, 6],
    'learning_rate': [0.1, 0.01],
    'n_estimators': [100, 200],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0]
}

grid = GridSearchCV(
    estimator=XGBClassifier(
        objective='multi:softprob',
        num_class=5,
        eval_metric='mlogloss',
        use_label_encoder=False,
        random_state=42
    ),
    param_grid=param_grid,
    cv=3,
    scoring='accuracy',
    verbose=1,
    n_jobs=-1
)

grid.fit(X_train, y_train)

print("Best Parameters:", grid.best_params_)
print("Best Score:", grid.best_score_)
```



# Modeling

## 4. XGBoost

```
# 최적 모델로 검증셋 평가
best_model = grid.best_estimator_
y_best_pred = best_model.predict(X_val)

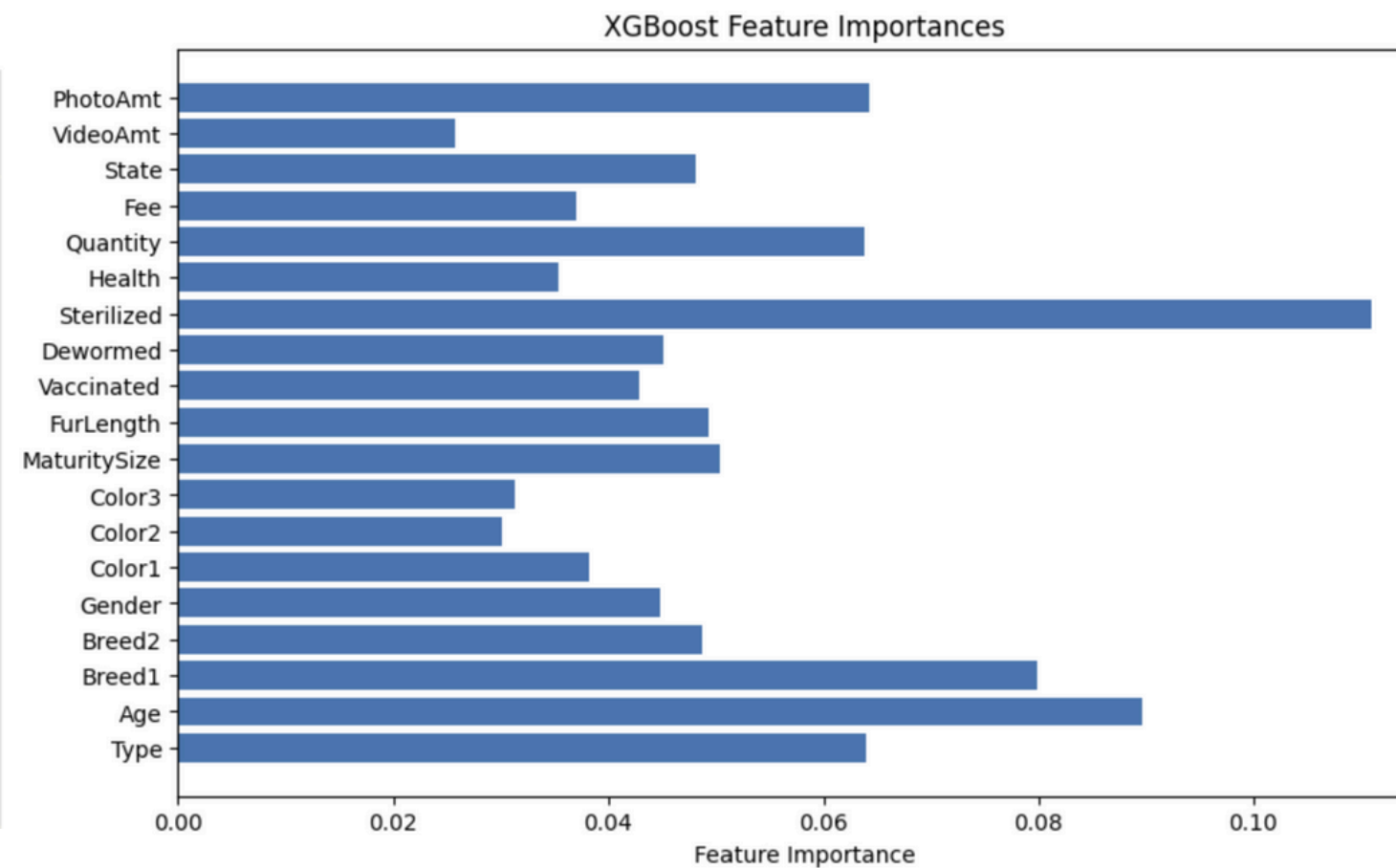
print(confusion_matrix(y_val, y_best_pred))
print(classification_report(y_val, y_best_pred))

xgb.feature_importances_
plt.figure(figsize=(10,6))
plt.barh(X.columns, best_model.feature_importances_)
plt.xlabel("Feature Importance")
plt.title("XGBoost Feature Importances")
plt.show()
```

# Modeling

## 4. XGBoost

| 기본 Accuracy | 최종 Accuracy | 상위 10개 중요 변수                                                                                                            |
|-------------|-------------|-------------------------------------------------------------------------------------------------------------------------|
| 0.3884      | 0.3897      | Breed1_307<br>Type<br>Age<br>Quantity<br>Breed2_307<br>Breed1_266<br>Breed1_285<br>Breed2_213<br>PhotoAmt<br>Breed1_243 |



# Modeling

## 5. Ensemble Methods

### [XGB+RF]

| 최종 Accuracy | 상위 10개 중요 변수                                                                                                               |
|-------------|----------------------------------------------------------------------------------------------------------------------------|
| 0.3954      | Breed1_307<br>Type<br>Age<br>Breed1_266<br>Breed1_285<br>Breed2_213<br>Breed1_292<br>Quantity<br>Breed2_307<br>Color_5_7_0 |

### [XGB+LGBM+Cat+B]

| 최종 Accuracy | 상위 10개 중요 변수                                                                                                               |
|-------------|----------------------------------------------------------------------------------------------------------------------------|
| 0.4071      | Breed1_307<br>Type<br>Age<br>Breed1_266<br>Breed1_285<br>Breed2_213<br>Breed1_292<br>Quantity<br>Breed2_307<br>Color_5_7_0 |

### [XGB+RF+LgR]

| 최종 Accuracy | 상위 10개 중요 변수                                                                                                     |
|-------------|------------------------------------------------------------------------------------------------------------------|
| 0.43        | DescLengthWordN<br>ameLength<br>Age<br>PhotoAmt<br>Vaccinated<br>Gender<br>Quantity<br>FurLength<br>MaturitySize |



# Model Analysis

PetFinder

# Model Analysis

## 〈최종 성능 비교〉

|                    | XGBoost                                                                                                                 | XGBoost+RandomF<br>orest                                                                                                   | XGBoost+RandomF<br>orest+LogisticRegr<br>ession                                                                            | XGB+LGBM+CatBo<br>ost                                                                                                      |
|--------------------|-------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| 기본 Accuracy        | 0.38846282094                                                                                                           | -                                                                                                                          | -                                                                                                                          | -                                                                                                                          |
| 최종 Accuracy        | 0.38979659886                                                                                                           | 0.39546515505168                                                                                                           | 0.392130710236745<br>56                                                                                                    | 0.40713571190396                                                                                                           |
| 상위 10개 <u>중요변수</u> | Breed1_307<br>Type<br>Age<br>Quantity<br>Breed2_307<br>Breed1_266<br>Breed1_285<br>Breed2_213<br>PhotoAmt<br>Breed1_243 | Breed1_307<br>Type<br>Age<br>Breed1_266<br>Breed1_285<br>Breed2_213<br>Breed1_292<br>Quantity<br>Breed2_307<br>Color_5_7_0 | Breed1_307<br>Type<br>Age<br>Breed1_266<br>Breed1_285<br>Breed2_213<br>Breed1_292<br>Quantity<br>Breed2_307<br>Color_5_7_0 | Breed1_307<br>Type<br>Age<br>Breed1_266<br>Breed1_285<br>Breed2_213<br>Breed1_292<br>Quantity<br>Breed2_307<br>Color_5_7_0 |



# Model Analysis

## <최종 성능 비교>

|                    | Logistic Regression                                                                             | RandomForest                                                                                                     | RF+logistic+XGB                                                                                                  | LightGBM                                                                                                                                                                      |
|--------------------|-------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 기본 Accuracy        | 0.3467822607                                                                                    | 0.4215                                                                                                           | -                                                                                                                | 0.39                                                                                                                                                                          |
| 최종 Accuracy        | 0.347449149                                                                                     | 0.42                                                                                                             | 0.43                                                                                                             | 0.3766                                                                                                                                                                        |
| 상위 10개 <u>중요변수</u> | Breed<br>Color<br>Type<br>Age<br>HasName<br>PhotoAmt<br>FurLength<br>Quantity<br>IsMixed<br>Fee | DescLengthWord<br>NameLength<br>Age<br>PhotoAmt<br>Vaccinated<br>Gender<br>Quantity<br>FurLength<br>MaturitySize | DescLengthWord<br>NameLength<br>Age<br>PhotoAmt<br>Vaccinated<br>Gender<br>Quantity<br>FurLength<br>MaturitySize | DescLengthWord<br>Age<br>NameLength<br>PhotoAmt<br>Vaccinated<br>Fee<br>Gender<br>Quantity<br>FurLength<br>MaturitySize<br>Breed1_307<br>IsMixed<br>Color_Other<br>Breed1_266 |

The background is white and decorated with various colorful paw prints in shades of blue, pink, yellow, and grey. There are also stylized wavy lines in pink and brown. At the bottom corners, there are large, stylized brown cat faces with white whiskers and bell-shaped noses. A dashed brown line forms a rectangular frame around the central text.

# Widget & Form Creation

PetFinder

# Widget & Form Creation

## 〈위젯 생성 과정〉

1. 변수 통계 정규화
2. 위젯 옵션 변수 개선
3. 위젯 정의
4. 예측 함수 생성
5. 예측 수행
6. 버튼 연결
7. 디스플레이 생성



# Widget & Form Creation

```
# 애플 기본 한글 폰트로 설정 (Mac용)
plt.rcParams['font.family'] = 'AppleGothic'
plt.rcParams['axes.unicode_minus'] = False

# 정규화 통계
age_mean = X['Age'].mean()
age_std = X['Age'].std()
quantity_mean = X['Quantity'].mean()
quantity_std = X['Quantity'].std()

def convert_years_to_scaled(age_years):
    return ((age_years * 12) - age_mean) / age_std

def convert_quantity_to_scaled(q):
    return (q - quantity_mean) / quantity_std

# 위젯 옵션 추출 개선
breed_dict = dict(zip(breed['BreedID'], breed['BreedName']))
breed_keys = sorted({breed_dict[b] for b in pet['Breed1'].unique() if b in breed_dict})
color_dict = dict(zip(color['ColorID'], color['ColorName']))
color_keys = sorted({color_dict[c] for c in pet['Color1'].unique() if c in color_dict})
```

```
# 위젯 정의
gender = widgets Dropdown(
    options=[('Male', -1.1388), ('Female', 0.3284), ('Mixed', 1.7956)],
    description='Gender')

fur = widgets Dropdown(
    options=[('Short', -0.7804), ('Medium', 0.8889), ('Long', 2.5582)],
    description='FurLength')

size = widgets Dropdown(
    options=[('Small', -1.5732), ('Medium', 0.2518), ('Large', 2.0769), ('XLarge', 3.9019)],
    description='Size')

vacc = widgets Dropdown(
    options=[('No', -1.0952), ('Yes', 0.4026), ('Not Sure', 1.9005)],
    description='Vaccinated')

age = widgets.FloatSlider(description='Age (years)', min=0.0, max=20.0, step=0.5, value=1)
quantity = widgets.IntSlider(description='Quantity', min=1, max=10, step=1, value=1)

type_dropdown = widgets Dropdown(
    options=[('Dog', 1), ('Cat', 2)], description='Type')

color_select = widgets Dropdown(options=color_keys, description='Color')
breed_select = widgets Dropdown(options=breed_keys, description='Breed1')
fee = widgets.IntText(value=0, description='Fee')
has_name = widgets Dropdown(
    options=[('No Name', 0), ('Name', 1)],
    description='HasName')

output = widgets.Output()
```



# Widget & Form Creation

```
# 예측 함수
def on_predict(b):
    input_row = pd.DataFrame([0]*X.shape[1], index=X.columns).T
    input_row['Gender'] = gender.value
    input_row['FurLength'] = fur.value
    input_row['MaturitySize'] = size.value
    input_row['Vaccinated'] = vacc.value
    input_row['Age'] = convert_years_to_scaled(age.value)
    input_row['Quantity'] = convert_quantity_to_scaled(quantity.value)
    input_row['Type'] = type_dropdown.value
    input_row['Fee'] = fee.value
    input_row['HasName_No Name'] = has_name.value

    # One-hot
    color_col = f'Color_{color_select.value}'
    breed_col = f'Breed1_{breed_select.value}'
    if color_col in input_row.columns:
        input_row[color_col] = 1
    if breed_col in input_row.columns:
        input_row[breed_col] = 1

    # 예측
    pred_proba = model.predict_proba(input_row)[0]
    pred_class = model.predict(input_row)[0]
    confidence = pred_proba[pred_class]
```

```
with output:
    clear_output()
    print(f"1) 입양 예측 확률: {confidence:.2%}")
    print(f"2) 예측 입양 속도: {pred_class} (0=매우 빠름 ~ 4=매우 느림)\n")
    for i, p in enumerate(pred_proba):
        print(f"속도 {i}일 확률: {p:.1%}")

    plt.figure(figsize=(6, 4))
    bars = plt.bar(range(5), pred_proba, color='skyblue')
    plt.xticks(range(5), [f"{i}" for i in range(5)])
    plt.title("입양 속도별 예측 확률")
    plt.ylabel("확률")
    plt.xlabel("입양 속도 (0~4)")
    for bar in bars:
        height = bar.get_height()
        plt.text(bar.get_x() + bar.get_width()/2, height + 0.01, f"{height:.1%}", ha=
    plt.ylim(0, 1)
    plt.tight_layout()
    plt.show()

# 버튼 연결
btn = widgets.Button(description='입양 예측 실행')
btn.on_click(on_predict)

# 디스플레이
display(widgets.VBox([
    gender, fur, size, vacc, age, quantity,
    type_dropdown, color_select, breed_select,
    fee, has_name, btn, output
]))
```



# Demo Video

**Animal Type:**

Dog

**Age:**

3

**Fee:**

0

**Has Name?**



**Gender:**

Male

**Maturitysize:**

Small

**Furlength:**

Short

**Vaccinated:**

# Demo Image

Animal Type:

Age:

Fee:

Has Name?  
☒

Gender:

Maturitysize:

Furlength:

Vaccinated:

Color:

Breed:

Upload Photo:  
 corgi.jpg



입양 예측 확률

같은 날에 입양: 3.8%

1주일 내에 입양: 26.7%

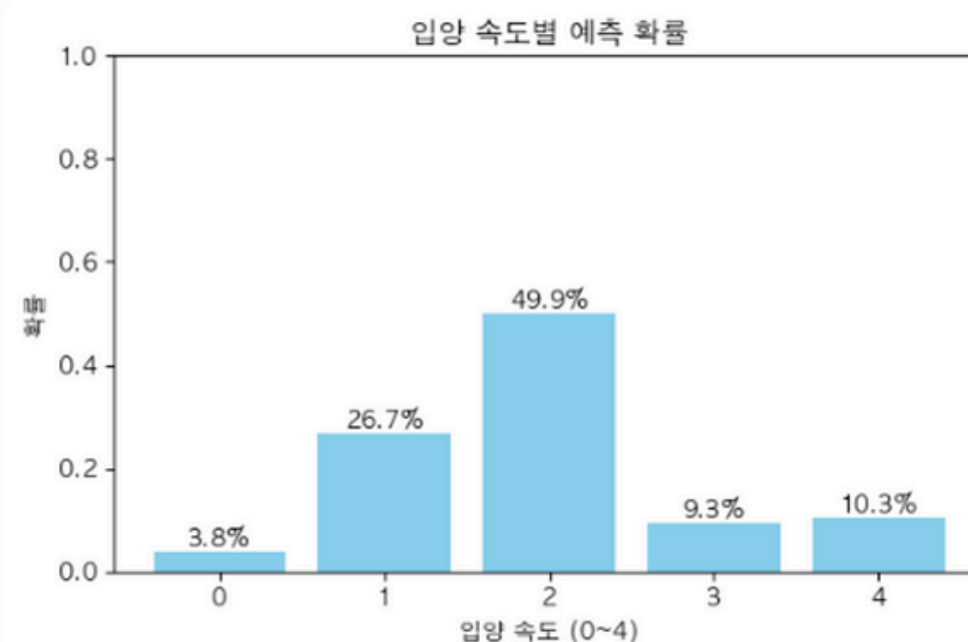
1달 내에 입양: 49.9%

2~3달 내에 입양: 9.3%

입양 안됨: 10.3%

예측 입양 속도: 1달 내에 입양

입양 속도별 예측 확률



다시 시도하기



Thank  
You